Corporación Unificada Nacional de Educación Superior · CUN

Escuela de Ingenierías · Ingeniería de Sistemas

ARQUITECTURA DE SOFTWARE (INGS0763)

Diseño de la arquitectura de un Sistema de Gestión Académica mediante Google AI Studio

Actividad de aprendizaje de las semanas 3, 4, 5 y 6

Estudiante	Juan Pablo Vargas Sánchez
Docente	Alexander Calderón Martínez
Fecha	8 de noviembre de 2026

Contenido

1. Introducción y descripción del problema 4
1.1 Problemas actuales y requerimientos que los atacan 4

2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5

3. Alcance, actores, supuestos y restricciones 6
3.1 Alcance 6
3.2 Actores 6
3.3 Supuestos 6
3.4 Restricciones 6

4. Requerimientos funcionales y no funcionales 8
4.1 Requerimientos funcionales 8
4.2 Requerimientos no funcionales 9
4.3 Reglas de negocio 10
4.4 Análisis de los requerimientos 10

5. Arquitectura seleccionada y justificación 12
5.1 La arquitectura en una frase 12
5.2 Alternativas evaluadas 12
5.3 Por qué monolito modular y no microservicios 12

6. Modelo del sistema 14
6.1 Modelo de casos de uso 14
6.2 Modelo de componentes (vista lógica) 17
6.3 Modelo de despliegue 21
6.4 Modelo de datos conceptual 22
6.5 Vista de secuencia: matrícula de un estudiante 26

7. Matriz de trazabilidad de requerimientos 29
7.1 Requerimientos funcionales 29
7.2 Requerimientos no funcionales 30
7.3 Cobertura 32

8. Decisiones arquitectónicas, atributos de calidad y análisis de riesgos 33
8.1 Decisiones arquitectónicas 33
8.2 Atributos de calidad y tácticas 33
8.3 Análisis de riesgos 33

9. Evidencias del uso de Google AI Studio 38
9.1 Configuración y método 38
9.2 Prompts utilizados 38
9.3 Resultado de la evaluación 39
9.4 Ajustes más importantes realizados a las propuestas de la IA 40
9.5 Conclusiones sobre el uso de la IA 41

10. Video de presentación y repositorio **43**

11. Conclusiones y recomendaciones **44**

11.1 Conclusiones 44

11.2 Recomendaciones 44

Referencias **44**

Anexo A. Evaluación detallada de las respuestas de la IA **46**

Anexo B. Capturas de la conversación en Google AI Studio **56**

Anexo C. Texto de los prompts **79**

1. Introducción y descripción del problema

Una institución de educación superior administra hoy la información de estudiantes, docentes, programas, asignaturas, grupos, matrículas, calificaciones y horarios en archivos y aplicaciones independientes. Esa dispersión genera tres problemas concretos: datos duplicados que se contradicen, consultas lentas y manuales, y la imposibilidad de saber quién cambió una nota, una matrícula o un rol, y cuándo.

Este informe presenta la **arquitectura de software de un Sistema de Gestión Académica (SGA)** que centraliza esa información en una sola fuente de verdad, accesible desde el navegador y controlada por roles. La propuesta parte de 12 requerimientos funcionales y 12 no funcionales, los traduce en criterios verificables y los resuelve con un estilo arquitectónico, cinco decisiones documentadas y seis modelos del sistema.

Google AI Studio (modelo Gemini 3.8 Flash) se usó como herramienta de apoyo en siete etapas del diseño. Cada propuesta de la IA se contrastó con los requerimientos y se aceptó, modificó o descartó con una justificación explícita (sección 9). La arquitectura final es responsabilidad del autor y no una transcripción de lo que sugirió la IA.

Alcance de la entrega. Siguiendo el enunciado, este trabajo produce una **arquitectura y sus modelos**, no una implementación. Todas las capacidades descritas son **propuestas de diseño**: ninguna se implementó ni se probó todavía. Para cada una, la matriz de trazabilidad indica cómo se verificaría.

1.1 Problemas actuales y requerimientos que los atacan

Tabla 1

Problemas actuales y requerimientos que los atacan

Problema actual	Consecuencia	Requerimientos que lo atacan
Datos duplicados en varias fuentes	Registros contradictorios de estudiantes, docentes y notas	RF-03, RF-04, RF-05, RNF-07
Consultas lentas y manuales	Estudiantes y coordinadores esperan respuestas que deberían ser inmediatas	RF-09, RF-10, RF-11, RNF-02
Sin trazabilidad	No se sabe quién cambió una nota, una matrícula o un rol, ni cuándo	RF-12, RNF-10

El SGA centraliza esta información en una sola fuente de verdad, con acceso web controlado por roles.

2. Objetivos

2.1 Objetivo general

Diseñar y justificar la arquitectura de software de un Sistema de Gestión Académica seguro, mantenible, escalable y accesible desde la web, a partir de sus requerimientos funcionales y no funcionales, usando Google AI Studio de forma crítica como herramienta de apoyo.

2.2 Objetivos específicos

1. Analizar el caso y especificar los requerimientos funcionales y no funcionales con criterios de aceptación verificables, junto con los actores, supuestos, restricciones y reglas de negocio.
2. Seleccionar y justificar un estilo arquitectónico frente al tamaño del sistema y sus atributos de calidad, comparando al menos tres alternativas.
3. Representar el sistema mediante modelos de casos de uso, componentes, despliegue, datos conceptual y secuencia, con nombres y relaciones consistentes entre sí.
4. Relacionar cada requerimiento con los componentes, modelos y decisiones que lo satisfacen, y con el método para verificarlo.
5. Identificar riesgos, puntos únicos de falla, problemas de seguridad y límites de escalabilidad, y proponer cómo validar las decisiones críticas.
6. Documentar el uso de Google AI Studio y evaluar cada una de sus propuestas como aceptada, modificada o descartada.

3. Alcance, actores, supuestos y restricciones

3.1 Alcance

Dentro del alcance: los 12 RF y 12 RNF del enunciado; el diseño de la arquitectura, sus modelos, decisiones, riesgos y método de verificación; la evaluación del uso de la IA.

Fuera del alcance: la implementación del sistema, la interfaz gráfica detallada, la integración real con otras plataformas institucionales (solo se deja la API preparada, RNF-08), los prerrequisitos entre asignaturas y las notificaciones por correo, que ningún requerimiento pide.

3.2 Actores

Tabla 2
Actores

Actor	Tipo	Qué hace en el sistema
Administrador	Principal	Gestiona usuarios, roles y permisos; registra y desactiva estudiantes y docentes; configura parámetros generales.
Coordinador académico	Principal	Gestiona programas, planes de estudio, asignaturas, periodos y grupos; asigna docentes; matricula estudiantes; genera reportes.
Docente	Principal	Consulta sus grupos y estudiantes; registra, actualiza y publica calificaciones.
Estudiante	Principal	Consulta sus asignaturas, horario y calificaciones publicadas.
Servicio de respaldo	Secundario (sistema)	Ejecuta copias de seguridad programadas de la base de datos.
Plataformas institucionales	Secundario (futuro)	Sistemas externos (financiero, LMS) que consumirán la API REST (RNF-08).

3.3 Supuestos

Lo que el caso no dice explícitamente y asumimos para poder diseñar:

Tabla 3
Lo que el caso no dice explícitamente y asumimos para poder diseñar

Id	Supuesto
S-01	"Personal autorizado" en RF-03 y RF-04 corresponde al rol **coordinador académico**, además del administrador.
S-02	La matrícula la registra el **coordinador académico**. Dejar que el estudiante se automatricule queda como evolución futura; la arquitectura lo permite sin cambios estructurales.
S-03	"Desactivar" (RF-03, RF-04) es una **baja lógica**: el registro cambia de estado y no se borra, para conservar el historial académico y la auditoría.
S-04	Las calificaciones tienen dos estados: **borrador** y **publicada**. El estudiante solo ve las publicadas (RF-09).

Id	Supuesto
S-05	Un usuario puede tener **uno o varios roles** (por ejemplo, un docente que también coordina) con una sola cuenta. *(Ajustado tras la evaluación de la IA, P4.)*
S-06	Escenario de referencia de RNF-02: **300 usuarios concurrentes**, con picos en semanas de matrícula y cierre de notas.
S-07	La institución opera una sola sede lógica; no se requiere despliegue en varias regiones.
S-08	El presupuesto es moderado: se prefieren servicios administrados o de código abierto frente a licencias costosas.
S-09	El estudiante no puede modificar ningún dato académico; solo consulta.

3.4 Restricciones

Tabla 4

Restricciones

Id	Restricción	Origen
R-01	La aplicación debe ser **web** y funcionar en navegadores recientes, en computador y móvil.	Contexto, RNF-06, RNF-12
R-02	Comunicación **solo por HTTPS**.	RNF-01
R-03	Contraseñas almacenadas con **hash y sal**, nunca en texto plano.	RNF-01
R-04	Interfaces expuestas como **API REST documentada**.	RNF-08
R-05	Disponibilidad mensual objetivo de **99,5 %** (máximo unas 3,6 horas de caída no programada al mes).	RNF-03
R-06	El alcance de esta actividad es **arquitectura y modelos**; no se exige implementar el sistema completo.	Enunciado
R-07	No usar datos personales reales ni credenciales en prompts, capturas o repositorios.	Enunciado, sección 10

4. Requerimientos funcionales y no funcionales

4.1 Requerimientos funcionales

Prioridad según el método MoSCoW: **M** = debe tener, **S** = debería tener.

Tabla 5

Requerimientos funcionales

Id	Requerimiento	Actor	Prioridad	Módulo responsable	Criterio de aceptación verificable
RF-01	Autenticación	Todos	M	Identidad y acceso	Con credenciales válidas se obtiene sesión; con inválidas se rechaza; al cerrar sesión el token deja de servir.
RF-02	Roles y permisos	Administrador	M	Identidad y acceso	Un estudiante que llama a un endpoint de docente recibe 403.
RF-03	Gestión de estudiantes	Administrador, coordinador	M	Gestión académica	Se crea, consulta, edita y desactiva un estudiante; el desactivado no aparece en la matrícula.
RF-04	Gestión de docentes	Administrador, coordinador	M	Gestión académica	Igual que RF-03 para docentes; el desactivado no se puede asignar a grupos.
RF-05	Programas y asignaturas	Coordinador	M	Gestión académica	No se aceptan dos asignaturas con el mismo código.
RF-06	Grupos y periodos	Coordinador	M	Gestión académica	Se crea un grupo con cupo, docente, horario y aula o enlace dentro de un periodo.
RF-07	Matrícula académica	Coordinador	M	Matrículas	Con cupo y sin duplicado, se matricula; sin cupo o duplicado, se rechaza con mensaje claro.
RF-08	Registro de calificaciones	Docente	M	Calificaciones	El docente registra notas solo en sus grupos; fuera de ellos recibe 403.
RF-09	Consulta de resultados	Estudiante	M	Calificaciones, Matrículas	El estudiante ve su horario y solo las notas publicadas.
RF-10	Reportes	Coordinador, administrador	S	Reportes	Reporte de matriculados filtrado por periodo y programa coincide con los datos de la base.
RF-11	Búsqueda y filtros	Usuarios autorizados	S	Transversal (cada módulo)	Buscar por identificador, nombre, programa, periodo o estado devuelve solo resultados permitidos para el rol.
RF-12	Auditoría	Administrador (consulta)	M	Auditoría	Cada inicio de sesión, cambio de rol, matrícula y cambio de nota genera un registro con usuario, fecha, acción y registro afectado.

4.2 Requerimientos no funcionales

Tabla 6

Requerimientos no funcionales

Id	Requerimiento	Atributo de calidad (ISO/IEC 25010)	Escenario o medida verificable	Táctica arquitectónica principal
RNF-01	Seguridad	Seguridad	100 % del tráfico por HTTPS; contraseñas con hash bcrypt o Argon2; prueba de acceso sin permiso devuelve 403.	TLS en el balanceador, hash de contraseñas, autorización por roles en la API.
RNF-02	Rendimiento	Eficiencia de desempeño	Con 300 usuarios concurrentes en carga normal, el percentil 95 de consultas comunes ≤ 3 s.	Índices en la base, paginación, consultas de reportes separadas, caché de catálogos.
RNF-03	Disponibilidad	Fiabilidad	Disponibilidad mensual ≥ 99,5 % sin contar mantenimientos programados.	Dos o más instancias de la API sin estado detrás de un balanceador; base de datos administrada con réplica.
RNF-04	Escalabilidad	Eficiencia de desempeño / Flexibilidad	Duplicar instancias de la API sin cambiar código.	API sin estado (sesión en token), escalado horizontal.
RNF-05	Mantenibilidad	Mantenibilidad	Cambiar una regla de calificaciones solo toca el módulo Calificaciones.	Monolito modular por contextos, capas y dependencias hacia interfaces.
RNF-06	Usabilidad y accesibilidad	Usabilidad	Funciones principales operables con teclado; contraste mínimo WCAG 2.1 AA.	Interfaz web responsiva con componentes accesibles.
RNF-07	Integridad y consistencia	Fiabilidad (integridad)	Prueba de concurrencia: 2 solicitudes simultáneas por el último cupo dejan 1 matrícula.	Transacción con bloqueo de fila sobre el grupo y restricción única en la base.
RNF-08	Interoperabilidad	Compatibilidad	Especificación OpenAPI publicada para todos los endpoints.	API REST documentada con OpenAPI.
RNF-09	Respaldo y recuperación	Fiabilidad (recuperabilidad)	Pérdida máxima de 15 min de datos (RPO) y restauración en menos de 2 h (RTO); simulacro de restauración trimestral exitoso.	Respaldo diario completo más archivado continuo del WAL, cifrado y fuera del centro de datos principal (ADR-003).
RNF-10	Trazabilidad	Mantenibilidad / Calidad del proceso	Cada RF y RNF aparece en la matriz con componente y método de verificación.	Matriz de trazabilidad y registros de decisiones (ADR).
RNF-11	Privacidad	Seguridad (confidencialidad)	Un estudiante no puede consultar datos de otro estudiante.	Filtros por propietario y alcance del rol en el dominio.
RNF-12	Compatibilidad	Compatibilidad / Portabilidad	Funciona en las dos últimas versiones de Chrome, Edge, Firefox y Safari, en 360 px y en 1366 px.	Interfaz web estándar y diseño responsivo.

4.3 Reglas de negocio

Reglas que el sistema debe validar en la capa de dominio (no solo en la interfaz):

Tabla 7

Reglas que el sistema debe validar en la capa de dominio (no solo en la interfaz)

Id	Regla	Requerimiento
RN-01	Un estudiante no puede matricularse **dos veces en el mismo grupo** ni en dos grupos de la **misma asignatura** en el mismo periodo.	RF-07
RN-02	No se puede matricular en un grupo **sin cupo disponible**. Con dos solicitudes simultáneas por el último cupo, solo una debe tener éxito.	RF-07, RNF-07
RN-03	Solo se matricula en grupos de un **periodo abierto** para matrícula.	RF-06, RF-07
RN-04	Solo se matriculan estudiantes **activos**. Un estudiante desactivado conserva su historial.	RF-03, RF-07
RN-05	Un docente solo registra o modifica calificaciones de **estudiantes matriculados en los grupos que tiene asignados**.	RF-08
RN-06	Las calificaciones deben estar dentro de la **escala** definida por la institución (supuesto: 0,0 a 5,0).	RF-08
RN-07	Toda modificación de una calificación **ya publicada** queda en auditoría con el valor anterior y el nuevo.	RF-08, RF-12
RN-08	El estudiante solo consulta **sus propios** datos; el docente, los de sus grupos; el coordinador, los de su programa.	RF-02, RNF-11
RN-09	El **código** de asignatura y el **identificador** de estudiante y docente son únicos.	RF-03, RF-04, RF-05
RN-10	Un grupo pertenece a una asignatura y a un periodo, y tiene un cupo máximo mayor que cero.	RF-06
RN-11	La matrícula y el registro de notas solo se permiten **dentro de las fechas** definidas para el periodo. Fuera de ellas, estudiantes y docentes solo consultan. *(Aporte de la IA, P1.)*	RF-06, RF-07, RF-08
RN-12	Cuando el coordinador **cierra las calificaciones** del periodo, el docente ya no puede editarlas. Un cambio posterior requiere autorización del administrador y queda en auditoría. *(Aporte de la IA, P1.)*	RF-08, RF-12
RN-13	Un estudiante no puede matricular dos grupos con **cruce de horario**, y un docente no puede tener asignados dos grupos en la misma franja. *(Aporte de la IA, P1.)*	RF-06, RF-07
RN-14	Los registros de auditoría son de **solo inserción**: nadie, ni el administrador, puede modificarlos ni borrarlos. *(Aporte de la IA, P1, ajustado.)*	RF-12, RNF-10
RN-15	En un grupo presencial, el **cupo no puede superar la capacidad del aula** asignada. *(Aporte de la IA, P7.)*	RF-06

4.4 Análisis de los requerimientos

- **Los RNF que más pesan en la arquitectura** son RNF-03 (disponibilidad), RNF-05 (mantenibilidad) y RNF-07 (consistencia). Son los que deciden el estilo arquitectónico; los demás se resuelven con tácticas puntuales.
- **RNF-02 pide un escenario de calidad completo.** Fuente: estudiantes y docentes. Estímulo: consultas comunes. Artefacto: API y base de datos. Entorno: operación normal con 300 usuarios concurrentes. Respuesta: devolver los datos. Medida: el 95 % en 3 s o menos.

- **RNF-09 no da cifras.** Se propone un RPO de 15 minutos y un RTO de 2 horas. La propuesta inicial era de 24 h y 4 h; se mejoró tras evaluar la respuesta P6 de la IA, porque los servicios administrados ya incluyen recuperación a un punto en el tiempo. La institución debe confirmar estos valores.
- **RNF-10 se cumple con el propio informe:** la matriz de trazabilidad y los ADR son su evidencia.

5. Arquitectura seleccionada y justificación

5.1 La arquitectura en una frase

Un **monolito modular en capas**: una sola aplicación desplegable, dividida en seis módulos por área de negocio (identidad y acceso, gestión académica, matrículas, calificaciones, reportes y auditoría). Cada módulo tiene tres capas internas (controlador, dominio y repositorio), se comunica con los demás solo por interfaces y es dueño de su propio esquema en una única base de datos PostgreSQL. El sistema se expone como **API REST** y se despliega en **dos nodos sin estado** detrás de un balanceador, con la base de datos en **alta disponibilidad** y respaldo externo continuo.

5.2 Alternativas evaluadas

Tabla 8

Alternativas evaluadas

Criterio	Capas sin módulos	Monolito modular (elegida)	Microservicios
Consistencia de la matrícula (RNF-07)	Transacción ACID local	Transacción ACID local	Consistencia eventual (Sagas); riesgo de sobrecupo
Rendimiento con 300 usuarios (RNF-02)	Suficiente	Llamadas en memoria, sin red	Latencia entre servicios
Disponibilidad (RNF-03)	2 réplicas + balanceador	2 réplicas + balanceador	Alta, pero con fallos en cascada
Mantenibilidad (RNF-05)	Se degrada con el tiempo	Módulos cohesivos con interfaces	Alta por servicio, pero con contratos distribuidos
Costo y operación	Bajo	Bajo: un solo artefacto	Alto: orquestador, trazas distribuidas

5.3 Por qué monolito modular y no microservicios

La decisión se tomó por los requerimientos, no por la tecnología de moda:

- **La consistencia de la matrícula (RNF-07) es el requisito más delicado.** Con una sola base de datos, el control de cupos y duplicados se resuelve con una transacción ACID. Con microservicios, cada servicio tendría su propia base y habría que coordinar con consistencia eventual (patrón Saga), lo que abre la puerta al sobrecupo.
- **300 usuarios concurrentes (RNF-02) es una carga moderada.** Las llamadas entre módulos ocurren en memoria, sin latencia de red; dos nodos bastan, y se puede agregar un tercero sin cambios (RNF-04).
- **La mantenibilidad (RNF-05) no exige microservicios.** Exige límites claros, y los módulos con interfaces y esquemas propios los dan, sin la complejidad operativa de orquestar decenas de servicios.
- **El contexto lo favorece:** presupuesto moderado (S-08) y un equipo pequeño.

Por qué no un monolito en capas sin módulos: las capas solas no impiden que la lógica de matrícula, notas y auditoría se mezcle con el tiempo. Los módulos agregan esos límites verticales.

Camino de evolución: si en el futuro un módulo necesita escalar por separado, ya tiene su interfaz y su esquema, de modo que puede extraerse como servicio de forma gradual (patrón Strangler Fig).

Google AI Studio recomendó el mismo estilo (respuesta P2). La sección 9 explica qué se tomó de su análisis y qué se corrigió: por ejemplo, que capas y módulos no son excluyentes sino complementarios.

6. Modelo del sistema

Los seis modelos usan los mismos nombres de actores, módulos, interfaces y entidades. Los diagramas se elaboraron en Mermaid y SVG; sus archivos fuente están en el repositorio (`modelos/fuentes/`).

6.1 Modelo de casos de uso

Notación UML: las figuras son los actores, las elipses los casos de uso y el rectángulo el límite del sistema. Una línea continua indica que el actor participa en el caso de uso. Una flecha punteada «include» indica que el caso de uso **siempre** ejecuta al caso incluido: CU-15 Registrar evento de auditoría es un caso interno, sin actor, que cumple RF-12.

Descripción de los casos de uso principales

Tabla 9

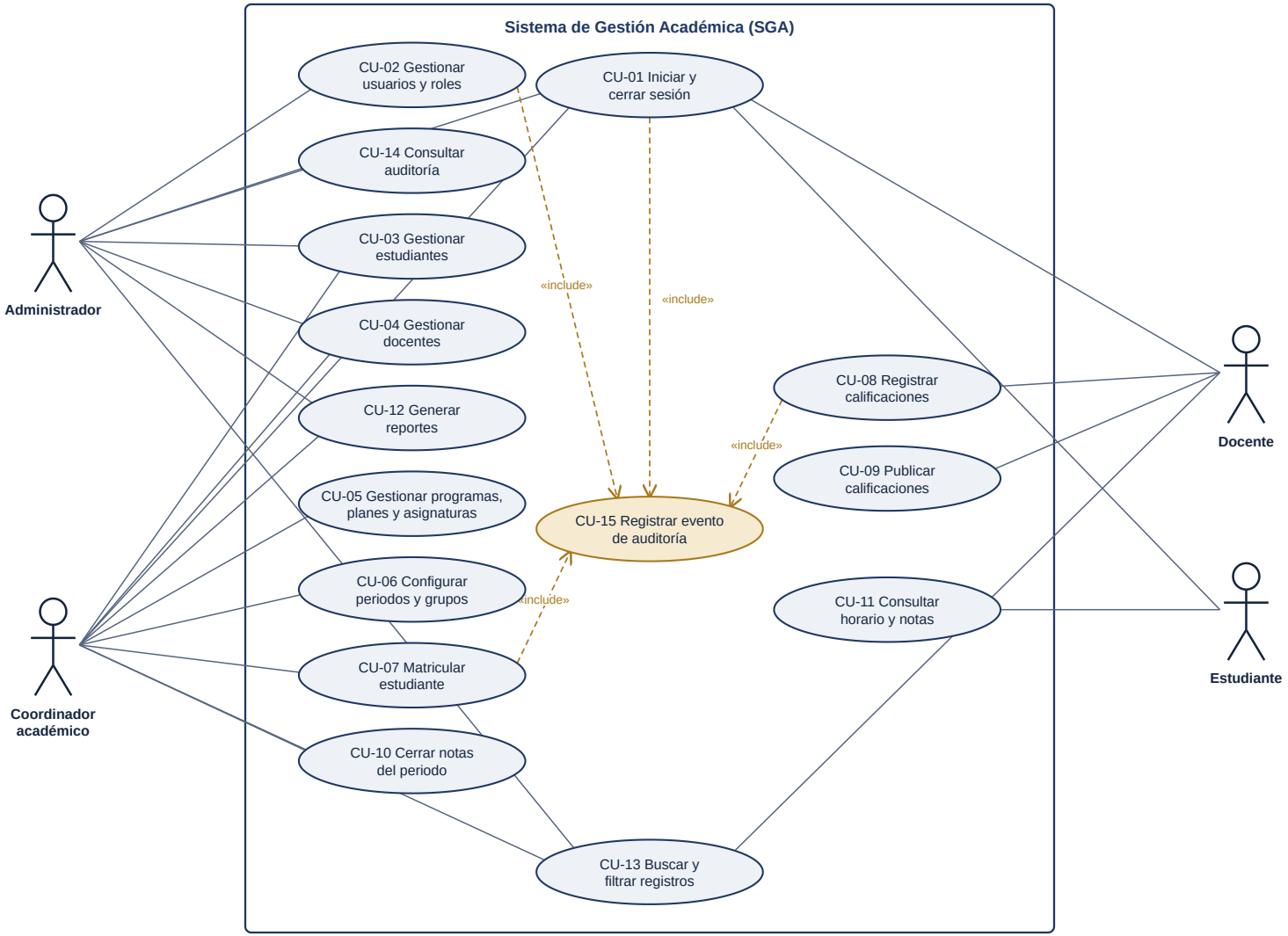
Descripción de los casos de uso principales

Id	Caso de uso	Actor(es)	Descripción breve	Precondición	Resultado	RF
CU-01	Iniciar y cerrar sesión	Todos	El usuario ingresa con su correo y contraseña; el sistema verifica el hash y entrega los tokens. Al cerrar sesión, revoca el token de renovación.	Cuenta activa	Sesión iniciada o cerrada; evento auditado	RF-01
CU-02	Gestionar usuarios y roles	Administrador	Crea cuentas, asigna o retira roles y desactiva usuarios.	Sesión de administrador	Roles actualizados; cambio auditado	RF-02
CU-03	Gestionar estudiantes	Administrador, coordinador	Registra, consulta, actualiza y desactiva estudiantes (baja lógica).	Programa existente	Estudiante activo o inactivo	RF-03
CU-04	Gestionar docentes	Administrador, coordinador	Igual que CU-03 para docentes.	—	Docente activo o inactivo	RF-04
CU-05	Gestionar programas, planes y asignaturas	Coordinador	Crea y actualiza programas, planes de estudio y asignaturas (código, nombre, créditos, semestre).	—	Oferta académica actualizada	RF-05
CU-06	Configurar periodos y grupos	Coordinador	Define el periodo y sus fechas; crea grupos con cupo, docente, horario y aula o enlace.	Asignatura y docente activos	Grupo listo para matrícula	RF-06
CU-07	**Matricular estudiante**	Coordinador	Inscribe a un estudiante en uno o varios grupos, validando cupo, duplicados, cruce de horario y fechas (ver Figura 6).	Periodo abierto, estudiante activo	Matrícula creada o rechazo con motivo; evento auditado	RF-07

Id	Caso de uso	Actor(es)	Descripción breve	Precondición	Resultado	RF
CU-08	**Registrar calificaciones**	Docente	Registra o corrige notas por evaluación en sus grupos, dentro de las fechas y antes del cierre.	Docente asignado al grupo	Notas en borrador; cambios auditados	RF-08
CU-09	Publicar calificaciones	Docente	Hace visibles las notas para los estudiantes.	Notas registradas	Notas publicadas	RF-08, RF-09
CU-10	Cerrar notas del periodo	Coordinador	Bloquea la edición de notas del periodo (RN-12).	Fecha límite cumplida	Notas congeladas	RF-08
CU-11	Consultar horario y notas	Estudiante	Consulta sus asignaturas, su horario y sus notas publicadas. Solo ve sus propios datos.	Estudiante matriculado	Información propia mostrada	RF-09
CU-12	Generar reportes	Coordinador, administrador	Reportes de matriculados, grupos, asignaturas y resultados con filtros por periodo y programa.	—	Reporte en pantalla o CSV	RF-10
CU-13	Buscar y filtrar registros	Administrador, coordinador, docente	Busca por identificador o nombre y filtra por programa, periodo o estado, según su rol.	—	Lista filtrada	RF-11
CU-14	Consultar auditoría	Administrador	Consulta los eventos por usuario, acción y rango de fechas.	—	Lista de eventos	RF-12
CU-15	Registrar evento de auditoría	(interno)	Guarda usuario, fecha, acción y registro afectado dentro de la misma transacción del cambio.	Incluido por CU-01, 02, 07 y 08	Evento inmutable	RF-12

Figura 1

Diagrama de casos de uso del SGA



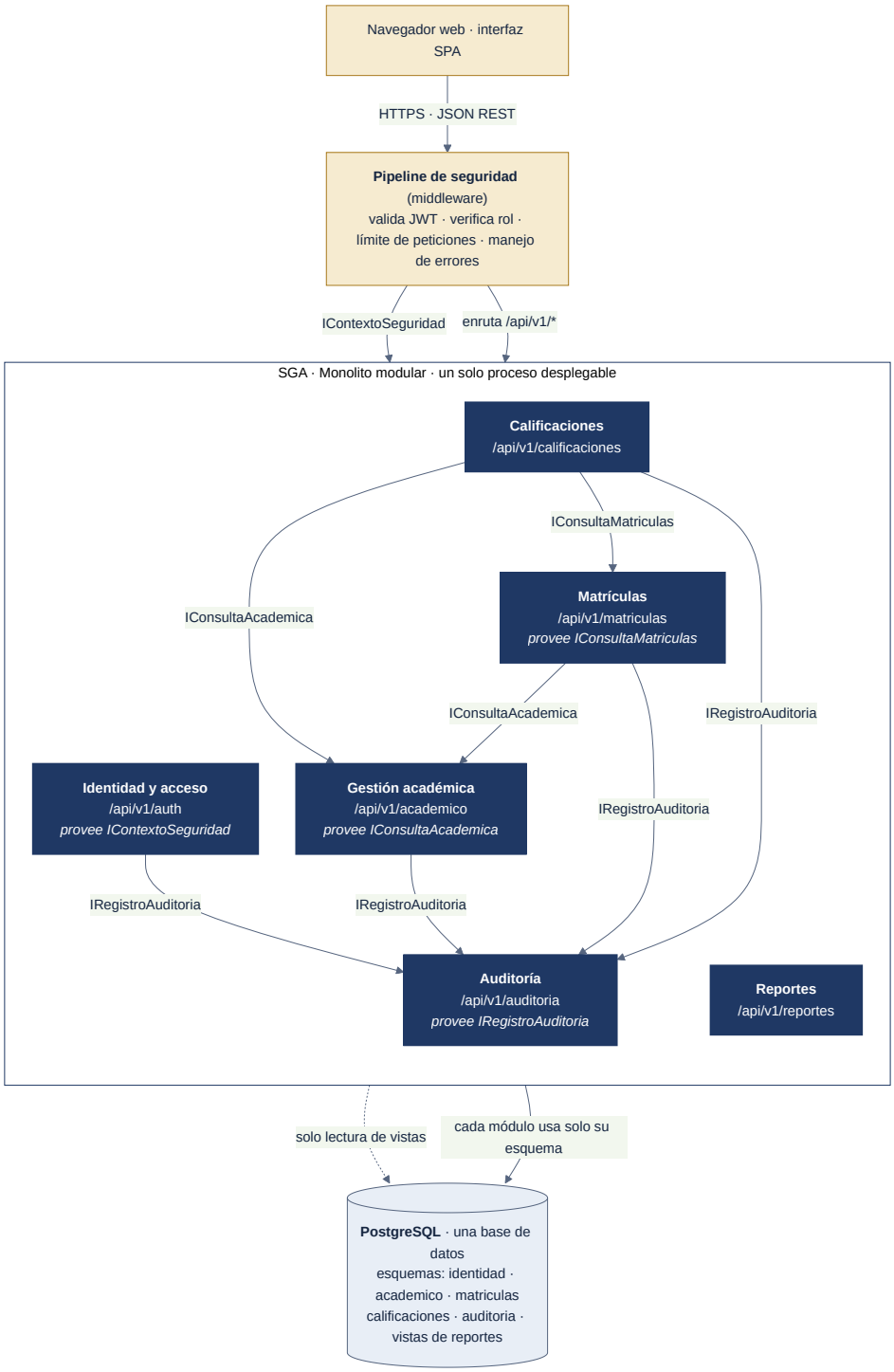
CU-15 es interno: lo incluyen los casos auditables (RF-12). Fuente: elaboración propia.

Nota. Las figuras son actores; las elipses, casos de uso; el rectángulo, el límite del sistema. Las flechas «include» indican que el caso siempre ejecuta CU-15 (auditoría, RF-12). Fuente: elaboración propia.

6.2 Modelo de componentes (vista lógica)

Figura 2

Módulos, interfaces y dependencias



Nota. Cada caja azul es un módulo con su ruta REST; la etiqueta de cada flecha es la interfaz que consume. Ningún módulo accede a las tablas de otro. Fuente: elaboración propia.

Cómo leerlo: cada caja azul es un módulo del monolito con su ruta REST. Una flecha indica que un módulo **usa** a otro, y la etiqueta es la **interfaz** que consume. Ningún módulo accede a las tablas de otro: solo a su propio esquema (ADR-001). Reportes es la única excepción controlada: lee vistas con un usuario de base de datos de solo lectura.

Responsabilidades e interfaces

Tabla 10

Responsabilidades e interfaces

Módulo	Responsabilidad	Endpoints REST principales	Provee	Depende de	RF / RNF
Identidad y acceso	Credenciales (hash Argon2id), inicio y cierre de sesión, emisión de tokens, usuarios, roles y permisos.	`POST /auth/iniciar-sesion` · `POST /auth/cerrar-sesion` · `POST /auth/renovar` · `PUT /auth/usuarios/{id}/rol`	`IContextoSeguridad` (usuario actual, rol)	`IRegistroAuditoria`	RF-01, RF-02, RNF-01, RNF-11
Gestión académica	Estudiantes, docentes, programas, planes, asignaturas, periodos, grupos, horarios y aulas o enlaces. Baja lógica. Cierre de notas del periodo.	`GET/POST/PUT /academico/estudiantes` · `…/docentes` · `…/programas` · `…/asignaturas` · `…/periodos` · `…/grupos` · `POST /academico/periodos/{id}/cerrar-notas`	`IConsultaAcademica` (¿estudiante activo?, datos del grupo, docente asignado, horario, estado del periodo)	`IRegistroAuditoria`	RF-03 a RF-06, RF-11
Matrículas	Matricular y anular con control de cupo, duplicados y cruce de horario en una transacción. Dueño de `cupo_grupo`.	`POST /matriculas` · `DELETE /matriculas/{id}` · `GET /matriculas?estudiante=&periodo=` · `GET /matriculas/mi-horario`	`IConsultaMatriculas` (matriculados de un grupo, matrículas de un estudiante)	`IConsultaAcademica`, `IRegistroAuditoria`	RF-07, RF-09, RNF-07
Calificaciones	Registrar, actualizar y publicar notas. Solo el docente del grupo, dentro de fechas y antes del cierre.	`GET /calificaciones/grupos/{id}` · `PUT /calificaciones/grupos/{id}` · `POST /calificaciones/grupos/{id}/publicar` · `GET /calificaciones/mis-notas`	—	`IConsultaAcademica`, `IConsultaMatriculas`, `IRegistroAuditoria`	RF-08, RF-09, RNF-07, RNF-11

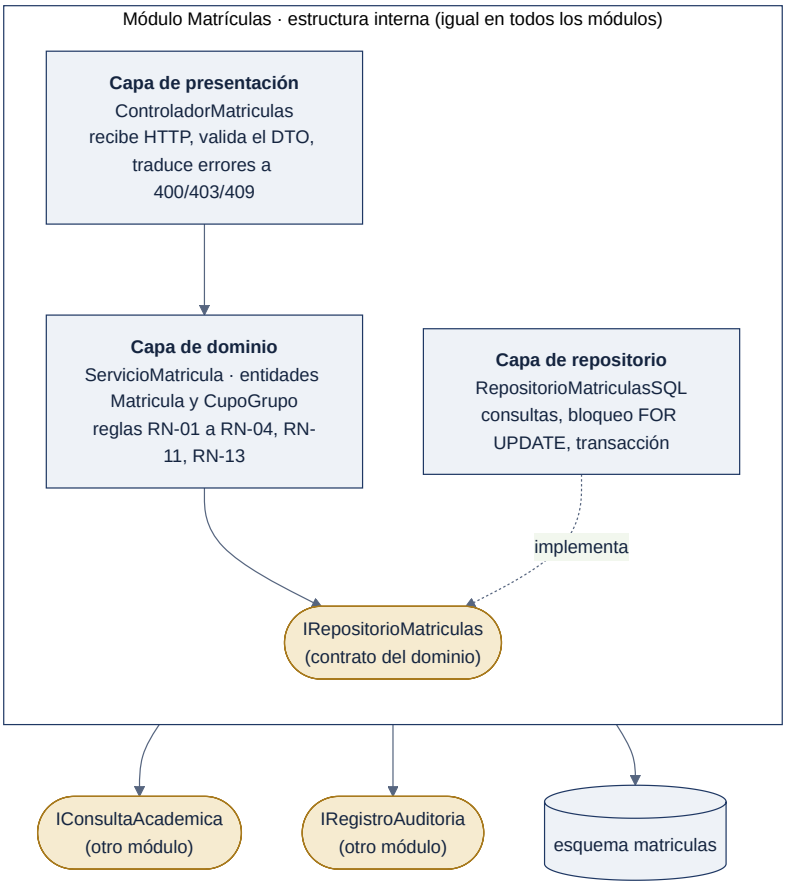
Módulo	Responsabilidad	Endpoints REST principales	Provee	Depende de	RF / RNF
Reportes	Reportes de matriculados, grupos, asignaturas y resultados con filtros por periodo y programa. Solo lectura.	`GET /reportes/matriculados?periodo=&programa=` · `GET /reportes/resultados?periodo=&programa=` · `GET /reportes/grupos?periodo=`	—	Vistas de solo lectura	RF-10, RF-11, RNF-02
Auditoría	Registrar eventos auditables de solo inserción y consultarlos (solo el administrador).	`GET /auditoria/eventos?usuario=&accion=&desde=&hasta=`	`IRegistroAuditoria`	—	RF-12, RNF-10

Búsqueda y filtros (RF-11): cada listado acepta `?q=` (identificador o nombre) y los filtros `programa`, `periodo` y `estado` que correspondan al módulo. El dominio aplica siempre el alcance del rol: un docente solo ve sus grupos y un estudiante solo sus datos (RNF-11).

Documentación de la API (RNF-08): todos los endpoints se describen en una especificación OpenAPI 3 (`api/openapi.yaml`).

Figura 3

Estructura interna de un módulo (Matrículas)



Nota. Todos los módulos siguen la misma estructura de tres capas, con el dominio dependiendo solo de contratos. Fuente: elaboración propia.

Todos los módulos siguen las mismas tres capas:

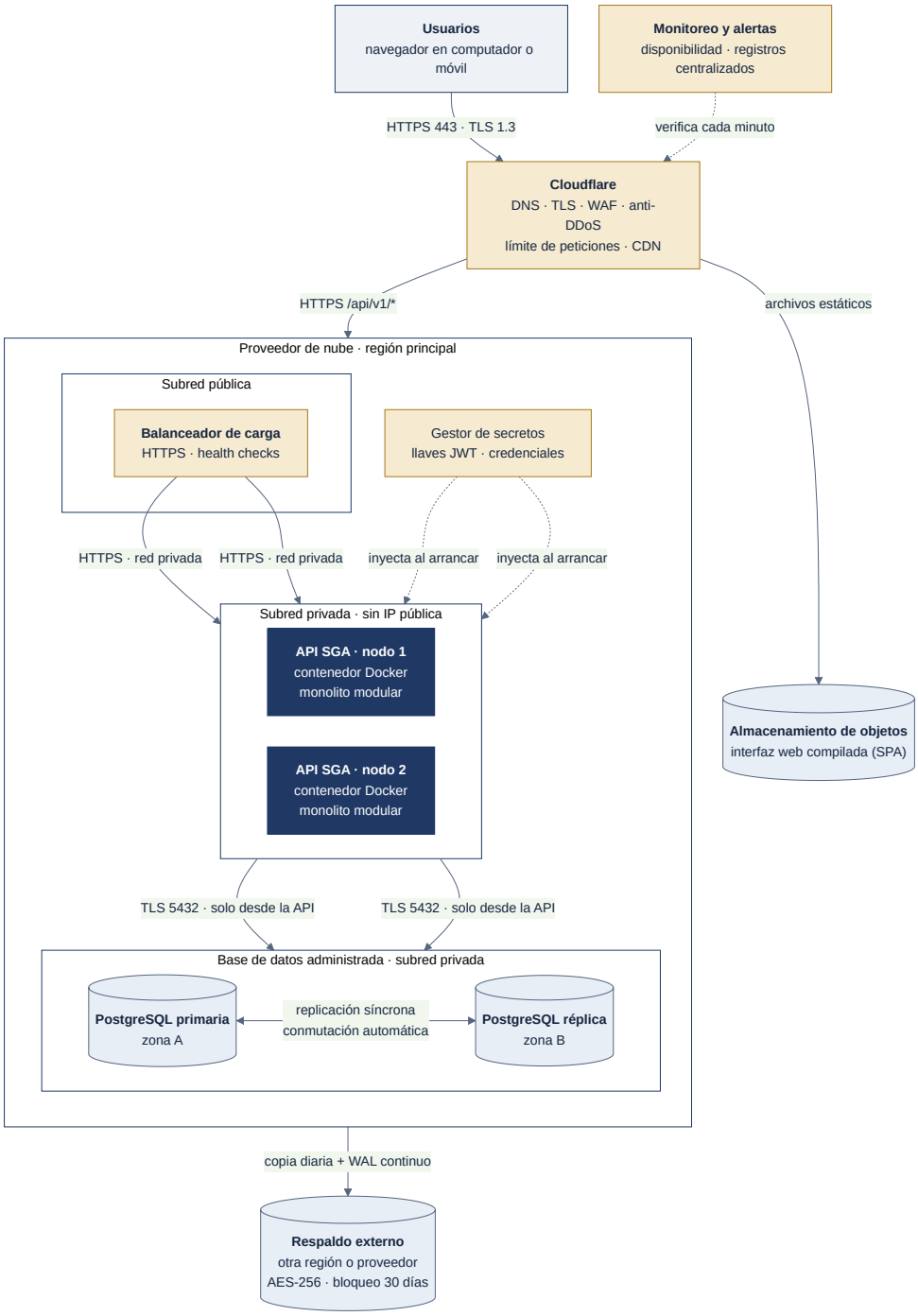
- **Presentación:** controlador REST. Recibe la petición, valida el formato del DTO y traduce errores del dominio a códigos HTTP (400 datos inválidos, 403 sin permiso, 409 conflicto, como "sin cupo").
- **Dominio:** servicios y entidades con las reglas de negocio. Depende solo de contratos (interfaces), nunca de la base de datos (principio de inversión de dependencias).
- **Repositorio:** implementa el contrato con SQL, maneja la transacción y el bloqueo.

Gracias a esta separación, cambiar el motor de base de datos o la interfaz no obliga a reescribir las reglas de negocio (RNF-05).

6.3 Modelo de despliegue

Figura 4

Nodos, canales y controles de seguridad



Nota. Las líneas continuas son canales de comunicación con su protocolo; las punteadas, dependencias operativas. Fuente: elaboración propia a partir de ADR-003.

Nodos

Tabla 11

Nodos

Nodo	Qué ejecuta	Por qué
Cloudflare	DNS, TLS, WAF, anti-DDoS, límite de peticiones y CDN	Primera línea de defensa y distribución de la interfaz sin costo adicional (RNF-01, RNF-02).
Almacenamiento de objetos	Interfaz web compilada (HTML, CSS, JS)	Archivos estáticos sin consumir CPU de la API (RNF-12).
Balanceador de carga administrado	Reparte el tráfico HTTPS entre los nodos y verifica su salud	Si un nodo falla, el tráfico pasa al otro en segundos. Al ser administrado, el proveedor lo mantiene redundante, así que no es un punto único de falla (RNF-03; crítica de la IA en P7).
API SGA · nodos 1 y 2	El monolito modular en contenedores Docker, sin estado	Redundancia y escalado horizontal: agregar un tercer nodo no requiere cambios (RNF-04, ADR-004).
PostgreSQL primaria y réplica	Base de datos administrada en dos zonas	Elimina el punto único de falla de la base de datos, con conmutación automática (RNF-03, RNF-07).
Respaldo externo	Copia diaria y WAL continuo, cifrados y con bloqueo de 30 días	RPO de 15 min y RTO de 2 h; protección contra *ransomware* (RNF-09).
Gestor de secretos	Llaves JWT y credenciales	Nada sensible en el repositorio (RNF-01).
Monitoreo y alertas	Verificación de disponibilidad y registros centralizados	Medir el 99,5 % y reaccionar a tiempo (RNF-03, RNF-10).

Controles de seguridad por canal

Tabla 12

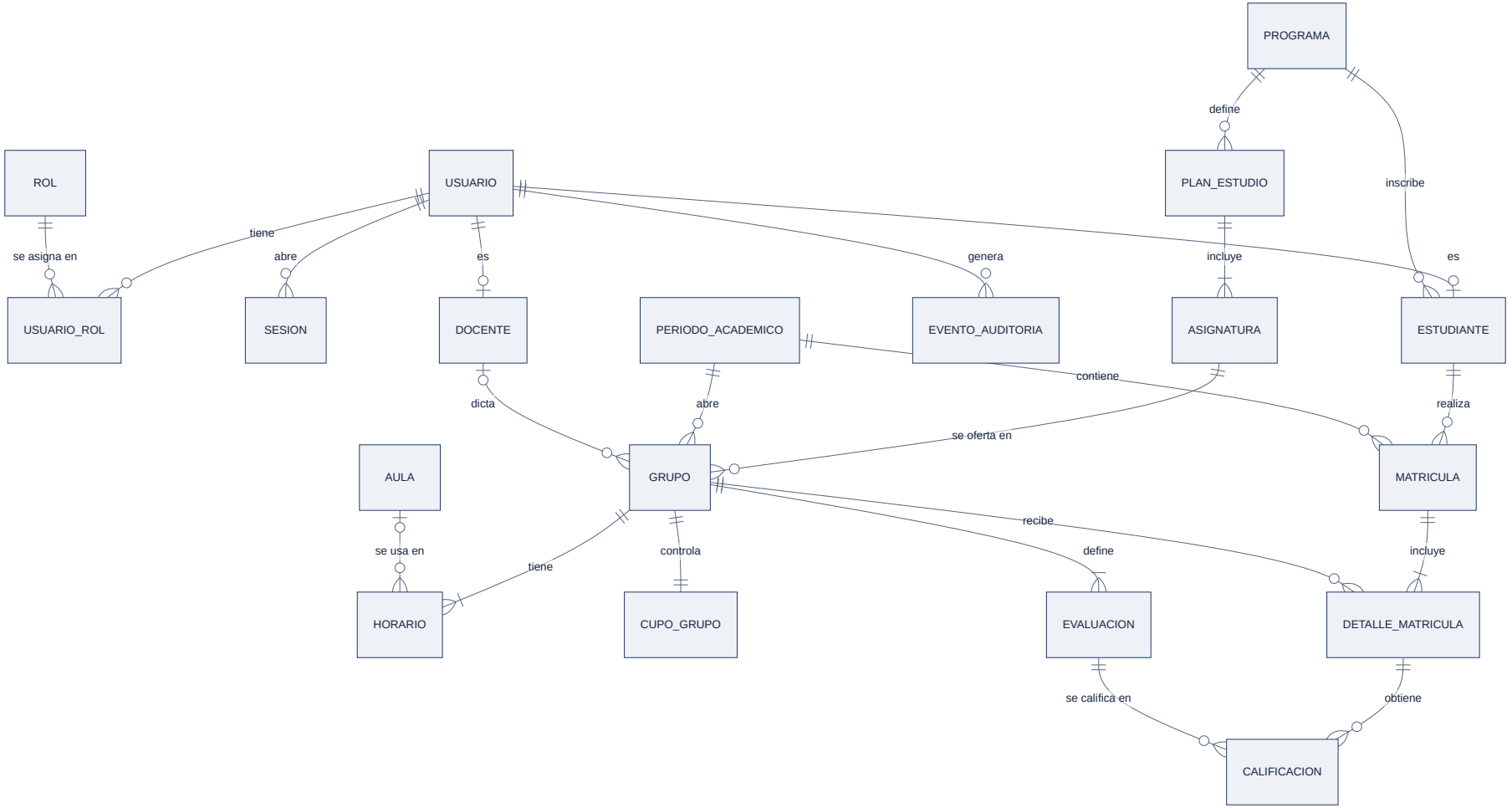
Controles de seguridad por canal

Canal	Protocolo	Control
Usuario → Cloudflare	HTTPS 443, TLS 1.3	Redirección forzada de HTTP a HTTPS; WAF y límite de peticiones.
Cloudflare → Balanceador	HTTPS	Solo se aceptan conexiones desde Cloudflare.
Balanceador → API	HTTPS en red privada	Los nodos no tienen IP pública.
API → PostgreSQL	TLS, puerto 5432	Subred privada; el puerto solo acepta tráfico de los nodos de la API; usuario de base de datos con permisos mínimos por esquema.
PostgreSQL → Respaldo	Cifrado AES-256	Almacenamiento en otra región o proveedor, con bloqueo de objetos.

6.4 Modelo de datos conceptual

Figura 5

Entidades principales y cardinalidades



Nota. Notación de pata de gallo: || uno y solo uno · o| cero o uno · |{ uno o muchos · o{ cero o muchos. Los atributos de cada entidad están en el modelo completo del repositorio (modelos/imagenes/04-datos-conceptual.png). Fuente: elaboración propia.

Entidades por módulo

Cada entidad pertenece al esquema del módulo que es su dueño (ADR-001).

Tabla 13

Entidades por módulo

Módulo (esquema)	Entidades
Identidad y acceso	Usuario, Rol, UsuarioRol, Sesion
Gestión académica	Estudiante, Docente, Programa, PlanEstudio, Asignatura, PeriodoAcademico, Grupo, Horario, Aula
Matrículas	Matricula, DetalleMatricula, CupoGrupo
Calificaciones	Evaluacion, Calificacion
Auditoría	EventoAuditoria

Cardinalidades más importantes

Tabla 14

Cardinalidades más importantes

Relación	Cardinalidad	Significado
Usuario – Rol	M:N (vía UsuarioRol)	Un usuario puede tener varios roles; por ejemplo, un docente que también coordina (S-05).
Usuario – Estudiante / Docente	1 : 0..1	Un usuario puede ser estudiante, docente, ambos o ninguno (administrador).
Programa – PlanEstudio – Asignatura	1:N y 1:N	Un programa tiene varios planes (versiones) y cada plan agrupa asignaturas con su semestre (RF-05).
Asignatura – Grupo	1:N	Una asignatura se ofrece en varios grupos por periodo.
PeriodoAcademico – Grupo	1:N	Cada grupo pertenece a un solo periodo.
Docente – Grupo	0..1 : N	Un grupo puede crearse sin docente y asignarse después (RF-06).
Grupo – Horario	1 : 1..N	Un grupo tiene una o más franjas semanales.
Aula – Horario	0..1 : N	Una franja presencial usa un aula; una virtual usa un enlace. El cupo del grupo no puede superar la capacidad del aula (RN-15, aporte de la IA en P7).
Grupo – CupoGrupo	1:1	Control de ocupación, propiedad del módulo Matrículas (ADR-002).
Estudiante – Matricula	1:N	Una matrícula por estudiante y periodo (restricción única).
Matricula – DetalleMatricula	1 : 1..N	Una matrícula incluye uno o más grupos.
Grupo – Evaluacion	1 : 1..N	Los cortes del grupo, cuyos porcentajes suman 100.
DetalleMatricula – Calificacion – Evaluacion	1:N y N:1	Una nota por estudiante matriculado y por evaluación (restricción única).

Restricciones de integridad (RNF-07)

Tabla 15

Restricciones de integridad (RNF-07)

Restricción	Protege la regla
`UNIQUE (id_estudiante, id_periodo)` en Matricula	Una sola matrícula por estudiante y periodo.
`UNIQUE (id_matricula, id_grupo)` en DetalleMatricula	No matricular dos veces el mismo grupo (RN-01).
`UNIQUE (id_matricula, id_asignatura)` en DetalleMatricula	No matricular dos grupos de la misma asignatura (RN-01).
FK compuesta `(id_grupo, id_asignatura)` → Grupo	El `id_asignatura` del detalle coincide con el del grupo.
`CHECK (ocupados <= cupo_maximo)` en CupoGrupo	Nunca hay sobrecupo, aunque falle la aplicación (RN-02).
`UNIQUE (id_detalle, id_evaluacion)` en Calificacion	Una sola nota por estudiante y evaluación.
`CHECK (nota BETWEEN 0 AND 5)` en Calificacion	Escala de notas (RN-06).
Solo INSERT y SELECT sobre EventoAuditoria	Auditoría inmutable (RN-14, ADR-005).
Baja lógica (`estado`) en Usuario, Estudiante y Docente	Se conserva el historial (S-03).

Atributos principales

Tabla 16

Atributos principales

Entidad	Atributos principales
Usuario	id_usuario (PK), correo (único), hash_contrasena (Argon2id), nombres, apellidos, estado
Rol / UsuarioRol	id_rol (PK), código (único: ADMIN, COORDINADOR, DOCENTE, ESTUDIANTE) / (id_usuario, id_rol)
Sesion	id_sesion (PK), id_usuario, hash del token de renovación, expira_en, revocada
Estudiante / Docente	id (PK), id_usuario (único), código (único), id_programa (solo estudiante), estado
Programa / PlanEstudio	código (único), nombre, nivel / id_programa, versión, vigente
Asignatura	id_asignatura (PK), id_plan, código (único), nombre, créditos, semestre
PeriodoAcademico	código (único), fechas de inicio y fin, inicio y fin de matrícula, límite de notas, notas_cerradas
Grupo	id_grupo (PK), id_asignatura, id_periodo, id_docente, código, cupo_maximo
Horario / Aula	día, hora de inicio y fin, id_aula o enlace / código, sede, capacidad
CupoGrupo	id_grupo (PK), cupo_maximo, ocupados (CHECK ocupados ≤ cupo_maximo)
Matricula / DetalleMatricula	id_estudiante, id_periodo, fecha, estado / id_matricula, id_grupo, id_asignatura, estado
Evaluacion / Calificacion	id_grupo, nombre, porcentaje / id_detalle, id_evaluacion, nota (0,0–5,0), estado (Borrador o Publicada)

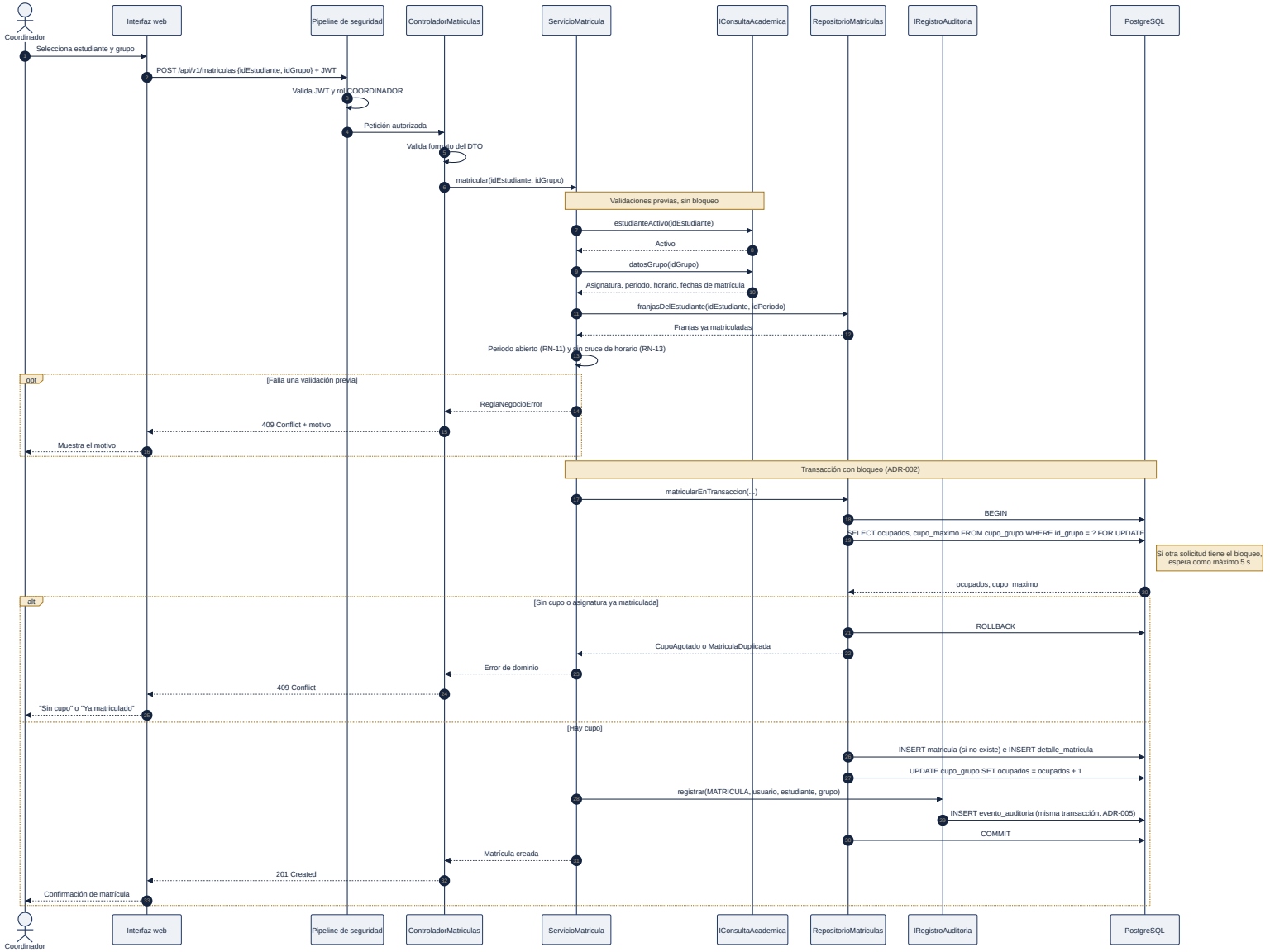
Entidad	Atributos principales
EventoAuditoria	id_usuario, acción, entidad, id del registro, valor anterior y nuevo, justificación, fecha y hora, IP, id de correlación

6.5 Vista de secuencia: matrícula de un estudiante

Se modela la matrícula porque es el proceso más crítico del sistema: combina reglas de negocio, concurrencia y auditoría (RF-07, RF-12, RNF-07).

Figura 6

Flujo completo de la matrícula



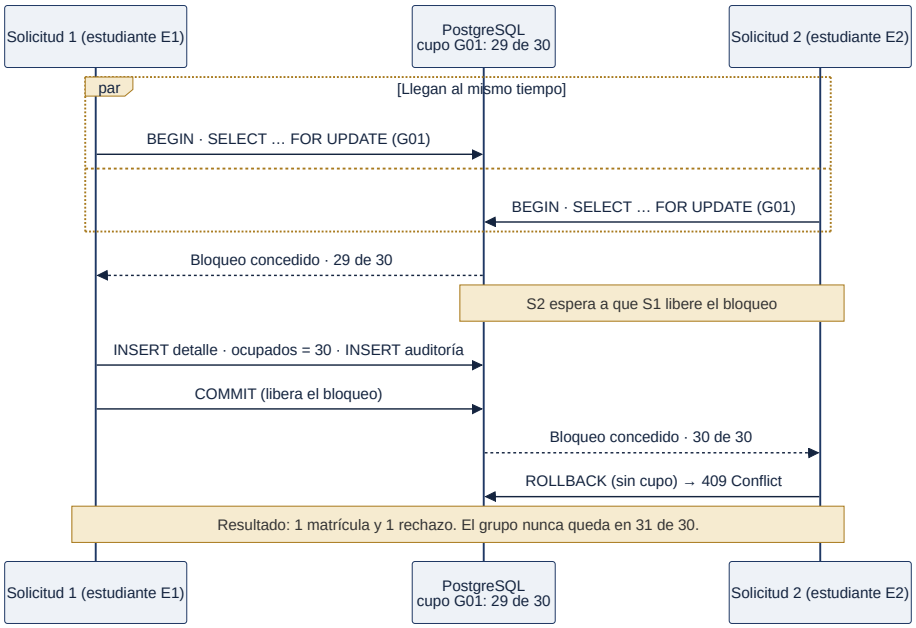
Nota. Los pasos 7 a 13 validan sin bloquear; los pasos 18 a 30 forman una sola transacción con bloqueo de fila (ADR-002) y auditoría (ADR-005). Fuente: elaboración propia.

Puntos clave del flujo:

1. **Pasos 2 a 5.** El pipeline de seguridad valida el token y el rol antes de que la petición llegue al módulo (RF-02, RNF-01).
2. **Pasos 7 a 13.** Las validaciones que no dependen del cupo se hacen **antes** del bloqueo: estudiante activo, periodo abierto y cruce de horario. Así el bloqueo dura lo mínimo.
3. **Paso 19.** `SELECT … FOR UPDATE` bloquea solo la fila del grupo en `cupo_grupo`. Otras matrículas de otros grupos siguen en paralelo (ADR-002).
4. **Pasos 26 a 30.** La matrícula, el cupo y el registro de auditoría se confirman en **una sola transacción**: o se guardan los tres o ninguno (ADR-005).
5. **Respuestas:** `201 Created` si se matricula; `409 Conflict` si no hay cupo, si hay un duplicado o si falla una regla de negocio.

Figura 7

Dos solicitudes simultáneas por el último cupo



Nota. Escenario de aceptación de RNF-07. Fuente: elaboración propia.

Este escenario es la prueba de aceptación de RNF-07. Se repite con 50 solicitudes simultáneas sobre un grupo de 1 cupo, y el resultado esperado es exactamente 1 matrícula.

Cuando una matrícula incluye varios grupos, los bloqueos se toman en **orden ascendente de `id_grupo`** para evitar interbloqueos entre dos solicitudes que comparten grupos (aporte de la IA, P5).

7. Matriz de trazabilidad de requerimientos

Cada requerimiento se relaciona con los componentes que lo satisfacen, los modelos donde aparece, las decisiones que lo sustentan y el método para verificarlo.

Abreviaturas de modelos: CU = casos de uso (Figura 1) · CO = componentes (Figuras 2 y 3) · DE = despliegue (Figura 4) · DA = datos (Figura 5) · SE = secuencia (Figuras 6 y 7).

Estado: como la actividad no exige implementar el sistema (restricción R-06), todos los requerimientos están **diseñados**, no probados. La columna de verificación indica cómo se comprobaría cada uno.

7.1 Requerimientos funcionales

Tabla 17

Requerimientos funcionales

Id	Requerimiento	Componente(s)	Modelos	Decisión / regla	Método de verificación y criterio de aceptación
RF-01	Autenticación	Identidad y acceso; pipeline de seguridad	CU-01 · CO · DA (Usuario, Sesion)	ADR-004	**Prueba de API:** credenciales válidas → 200 con tokens; inválidas → 401; tras cerrar sesión, renovar el token → 401.
RF-02	Roles y permisos	Identidad y acceso (UsuarioRol); pipeline; dominio de cada módulo	CU-02 · CO · DA (Rol, UsuarioRol)	ADR-004 · RN-08	**Matriz de permisos automatizada:** cada endpoint se llama con los 4 roles; los no autorizados reciben 403.
RF-03	Gestión de estudiantes	Gestión académica	CU-03 · CO · DA (Estudiante)	S-03 (baja lógica)	**Prueba funcional:** crear, consultar, editar y desactivar; el estudiante desactivado no se puede matricular.
RF-04	Gestión de docentes	Gestión académica	CU-04 · CO · DA (Docente)	S-03	**Prueba funcional:** igual que RF-03; un docente desactivado no se puede asignar a un grupo.
RF-05	Programas y asignaturas	Gestión académica	CU-05 · DA (Programa, PlanEstudio, Asignatura)	RN-09	**Prueba funcional:** crear un plan con asignaturas; un código de asignatura repetido → 409.
RF-06	Grupos y periodos	Gestión académica; Matrículas (CupoGrupo)	CU-06 · DA (PeriodoAcademico, Grupo, Horario, Aula, CupoGrupo)	RN-10 · RN-13 · RN-15	**Prueba funcional:** crear un grupo con cupo, docente, horario y aula o enlace; asignar al mismo docente en una franja cruzada → 409; cupo de 40 en un aula de 25 → 409.
RF-07	Matrícula académica	Matrículas; IConsultaAcademica; Auditoría	CU-07 · SE (Fig. 6 y 7) · DA (Matricula, DetalleMatricula)	ADR-002 · RN-01 a RN-04 · RN-11 · RN-13	**Pruebas de dominio** de cada regla y **prueba de concurrencia** (ver RNF-07).

Id	Requerimiento	Componente(s)	Modelos	Decisión / regla	Método de verificación y criterio de aceptación
RF-08	Registro de calificaciones	Calificaciones; IConsultaAcademica; IConsultaMatriculas	CU-08 · CU-10 · DA (Evaluacion, Calificacion)	RN-05 · RN-06 · RN-12	**Prueba de API:** un docente registra notas en su grupo → 200; en un grupo ajeno → 403; después del cierre → 409; nota de 5,5 → 400.
RF-09	Consulta de resultados	Calificaciones; Matrículas	CU-09 · CU-11 · DA (Calificacion.estado)	S-04 · RN-08	**Prueba de API:** el estudiante ve solo sus notas **publicadas**; las notas en borrador no aparecen.
RF-10	Reportes	Reportes (vistas de solo lectura)	CU-12 · CO	Evaluación IA 3.11	**Prueba con datos de ejemplo:** el total del reporte de matriculados por periodo y programa coincide con un conteo directo en la base.
RF-11	Búsqueda y filtros	Listados de cada módulo	CU-13 · CO	Evaluación IA 3.12 · RNF-11	**Prueba de API:** buscar por código, nombre, programa, periodo y estado; un docente solo obtiene resultados de sus grupos.
RF-12	Auditoría	Auditoría (IRegistroAuditoria)	CU-14 · CU-15 · SE (Fig. 6, pasos 28–29) · DA (EventoAuditoria)	ADR-005 · RN-07 · RN-14	**Prueba de integración:** cada inicio de sesión, cambio de rol, matrícula y cambio de nota genera un evento completo. **Prueba de caos:** matar el proceso (`kill -9`) durante un flujo de matrículas y comprobar que el número de matrículas coincide con el de eventos de auditoría.

7.2 Requerimientos no funcionales

Tabla 18

Requerimientos no funcionales

Id	Requerimiento	Componente(s)	Modelos	Decisión	Método de verificación y criterio de aceptación
RNF-01	Seguridad	Cloudflare; balanceador; pipeline; Identidad y acceso; gestor de secretos	DE · CO · DA (hash_contrasena)	ADR-003 · ADR-004	**Escaneo** con OWASP ZAP sin hallazgos altos; HTTP redirige a HTTPS; revisar que la tabla de usuarios solo tenga hashes Argon2id; análisis del repositorio sin secretos.
RNF-02	Rendimiento	API (2 nodos); índices; caché en memoria; pools de conexiones separados para reportes	DE · CO	ADR-001 · ADR-002 · ADR-003	**Prueba de carga** con k6: mezcla de 70 % consultas, 20 % matrículas y 10 % reportes, subiendo de 50 a 500 usuarios. Con 300, el percentil 95 de las consultas comunes debe ser ≤ 3 s; se registra el punto donde se degrada.

Id	Requerimiento	Componente(s)	Modelos	Decisión	Método de verificación y criterio de aceptación
RNF-03	Disponibilidad	Balanceador administrado + 2 nodos; PostgreSQL con réplica; monitoreo	DE	ADR-003	**Prueba de falla:** apagar un nodo y forzar la conmutación de la base; el servicio responde. **En operación:** monitor externo con disponibilidad mensual ≥ 99,5 %.
RNF-04	Escalabilidad	API sin estado	DE · CO	ADR-001 · ADR-004	**Prueba:** agregar un tercer nodo detrás del balanceador sin cambiar código y repetir la prueba de carga.
RNF-05	Mantenibilidad	Monolito modular en capas; interfaces entre módulos	CO (Fig. 2 y 3)	ADR-001	**Pruebas de arquitectura** (por ejemplo, ArchUnit) que fallan si un módulo accede al repositorio de otro; complejidad ciclomática < 10 medida con SonarQube.
RNF-06	Usabilidad y accesibilidad	Interfaz web; pipeline de integración continua	DE	—	**Automática en cada cambio:** axe-core y Lighthouse CI con puntaje de accesibilidad ≥ 90. **Manual:** recorrido completo de matrícula y registro de notas solo con teclado.
RNF-07	Integridad y consistencia	Matrículas (CupoGrupo, transacción); restricciones de la base; Calificaciones	SE (Fig. 6 y 7) · DA (restricciones)	ADR-002 · ADR-005	**Prueba de concurrencia:** 50 solicitudes simultáneas por un grupo de 1 cupo → exactamente 1 matrícula y 49 rechazos 409; `ocupados` nunca supera `cupo_maximo`.
RNF-08	Interoperabilidad	API REST `/api/v1`	CO	—	**Especificación OpenAPI** (`api/openapi.yaml`) validada con un linter; los endpoints responden según el contrato.
RNF-09	Respaldo y recuperación	PostgreSQL administrado; respaldo externo	DE	ADR-003	**Simulacro trimestral** de restauración en un entorno de pruebas: RPO real ≤ 15 min y RTO real ≤ 2 h.
RNF-10	Trazabilidad	Esta matriz; ADR; auditoría con identificador de correlación	Todos	ADR-001 a ADR-005	**Revisión documental:** cada RF y RNF tiene al menos un componente, un modelo y un método de verificación (esta tabla).
RNF-11	Privacidad	Dominio de cada módulo (filtros por propietario); subred privada	CO · DE	ADR-004 · RN-08	**Prueba de API:** el estudiante A pide los datos del estudiante B → 403; un docente consulta un grupo ajeno → 403.
RNF-12	Compatibilidad	Interfaz web responsiva	DE	—	**Pruebas en navegadores** con Playwright: Chrome, Edge, Firefox y Safari (WebKit), en 360 px y en 1366 px.

7.3 Cobertura

- **12 de 12** RF y **12 de 12** RNF tienen componente, modelo y método de verificación.
- Los requerimientos **más cubiertos** son RF-07, RF-12 y RNF-07: aparecen en 3 o más modelos y en 2 decisiones.
- Los **menos sustentados** en el diseño son RNF-06 y RNF-12: dependen de la implementación de la interfaz, que no está dentro del alcance. Se registran como riesgo de validación.

8. Decisiones arquitectónicas, atributos de calidad y análisis de riesgos

8.1 Decisiones arquitectónicas

Las decisiones se documentaron con el formato de registro de decisiones arquitectónicas (ADR) propuesto por Nygard (2011): contexto, decisión, opciones consideradas y consecuencias. El enunciado pide al menos tres; se documentaron cinco. El texto completo de cada una está en el repositorio (`docs/adr/`).

Tabla 19

Decisiones arquitectónicas

ADR	Decisión	Requerimientos	Alternativas descartadas	Consecuencia principal
ADR-001	**Monolito modular en capas**, un esquema por módulo, API REST	RNF-02, 03, 04, 05, 07, 09	Capas sin módulos; microservicios	Consistencia ACID en un solo artefacto; el sistema escala completo, no por módulo
ADR-002	**Bloqueo pesimista** de la fila `cupo_grupo` en una transacción, en orden ascendente de grupo, con validaciones previas sin bloqueo y restricciones únicas	RF-07, RNF-07, RNF-02	Control optimista; cola de solicitudes	Nunca hay sobrecupo; un grupo muy popular hace fila
ADR-003	**Dos nodos sin estado**, balanceador administrado, **PostgreSQL con réplica** en otra zona, respaldo continuo (RPO 15 min, RTO 2 h), Cloudflare	RNF-01, 03, 04, 09	Un solo servidor; Kubernetes multirregión	Sin puntos únicos de falla internos; dependencia de proveedores externos
ADR-004	**Autenticación sin estado:** JWT de 15 min, renovación de 8 h con rotación, rol activo único	RF-01, RF-02, RNF-01, 04, 11	Sesión en memoria; lista negra en Redis	Cualquier nodo atiende cualquier petición; el token sigue válido hasta 15 min tras cerrar sesión
ADR-005	**Auditoría en la misma transacción** del cambio, tabla de solo inserción, exportación diaria	RF-12, RNF-07, RNF-10	Bus de eventos asíncrono; almacenamiento WORM	Ningún cambio queda sin auditar; la tabla crece y se particiona por mes

8.2 Atributos de calidad y tácticas

Tabla 20

Atributos de calidad y tácticas

Atributo (ISO/IEC 25010)	Requerimiento	Escenario de calidad (estímulo → medida)	Táctica arquitectónica
Eficiencia de desempeño	RNF-02	300 usuarios concurrentes en operación normal → P95 de consultas comunes ≤ 3 s	Llamadas en memoria, índices, caché de catálogos, pool separado para reportes

Atributo (ISO/IEC 25010)	Requerimiento	Escenario de calidad (estímulo → medida)	Táctica arquitectónica
Fiabilidad: disponibilidad	RNF-03	Falla de un nodo o de la base primaria → el servicio sigue; disponibilidad mensual ≥ 99,5 %	Redundancia activa, conmutación automática, despliegues sin interrupción
Fiabilidad: integridad	RNF-07	50 solicitudes simultáneas por 1 cupo → exactamente 1 matrícula	Transacción con bloqueo de fila y restricciones de base de datos
Fiabilidad: recuperabilidad	RNF-09	Pérdida de la base de datos → restaurada en ≤ 2 h con pérdida ≤ 15 min	Respaldo diario + WAL continuo, almacenamiento externo con bloqueo
Seguridad	RNF-01, RNF-11	Usuario sin permiso pide un recurso → 403; tráfico siempre cifrado	TLS, Argon2id, RBAC más filtros por propietario, red privada
Mantenibilidad	RNF-05	Cambiar una regla de notas → solo se modifica el módulo Calificaciones	Módulos con interfaces, inversión de dependencias, pruebas de arquitectura
Compatibilidad	RNF-08, RNF-12	Integración externa o navegador distinto → funciona según contrato	API REST con OpenAPI; interfaz web responsiva

8.3 Análisis de riesgos

Este análisis combina la revisión propia con la autocrítica que se pidió a Google AI Studio (P7). Cada riesgo se califica con **Riesgo = Probabilidad × Impacto**, en una escala de 1 (bajo) a 3 (alto):

- **Nivel alto:** 6 a 9.
- **Nivel medio:** 3 o 4.
- **Nivel bajo:** 1 o 2.

Puntos únicos de falla

Tabla 21

Puntos únicos de falla

Componente	¿Es punto único de falla?	Cómo se resolvió
Nodo de la API	No	Dos nodos sin estado detrás del balanceador (ADR-003, ADR-004).
Balanceador de carga	No	Se usa el balanceador administrado del proveedor, redundante por diseño (crítica de la IA en P7).
Base de datos	No	Primaria y réplica en otra zona, con conmutación automática (ADR-003).
Redis	No existe	Se descartó: el diseño no depende de una caché compartida (evaluación P2, P3, P6).
Auditoría por bus de eventos	No existe	La auditoría va en la misma transacción del cambio (ADR-005).
Cloudflare	**Sí, residual**	Un proveedor externo de borde. Se acepta por su disponibilidad histórica alta y su costo cero; si cae, se puede apuntar el DNS directo al balanceador (procedimiento manual).
Región del proveedor de nube	**Sí, residual**	Todo corre en una región. Una caída regional completa supera el 99,5 %; se recupera desde el respaldo externo en otra región (RTO 2 h). Desplegar en varias regiones no se justifica por costo (S-08).

Registro de riesgos

Tabla 22

Registro de riesgos

Id	Riesgo	Tipo	P	I	Nivel	Mitigación en el diseño	Validación
R-01	Sobrecupo o matrícula duplicada por solicitudes simultáneas	Integridad	3	3	**9 Alto**	Bloqueo de fila en `cupo_grupo`, restricciones únicas y `CHECK` (ADR-002)	Prueba de concurrencia: 50 solicitudes por 1 cupo y 300 usuarios por 5 cupos
R-02	Cambio de nota o matrícula sin registro de auditoría	Trazabilidad	2	3	**6 Alto**	Auditoría en la misma transacción (ADR-005)	Prueba de caos con `kill -9`: matrículas = eventos
R-03	Un superusuario de la base de datos altera la auditoría	Seguridad	1	3	3 Medio	Usuario de aplicación sin UPDATE/DELETE; exportación diaria con bloqueo de objetos; registro de actividad del proveedor	Comparar la exportación con la tabla en un muestreo mensual
R-04	Cola de bloqueos supera 3 s si cientos de estudiantes piden el mismo grupo (si se habilita la automatrícula)	Rendimiento	2	2	4 Medio	Hoy matricula el coordinador (S-02); lectura previa sin bloqueo para rechazo rápido; `lock_timeout` de 5 s	Prueba extrema de 300 usuarios sobre 5 cupos en 2 s; medir P95 y P99
R-05	Agotamiento de conexiones a PostgreSQL	Rendimiento / Disponibilidad	2	3	**6 Alto**	Pool de 20 por nodo, *pooler* administrado, pool aparte para reportes	Prueba de carga de 50 a 500 usuarios; observar conexiones activas
R-06	Reportes pesados degradan la matrícula en el cierre de periodo	Rendimiento	2	2	4 Medio	Vistas de solo lectura, pool separado con tiempo máximo; réplica de lectura si hace falta	Prueba de carga con 10 % de reportes
R-07	Usuario con varios roles usa permisos de otro perfil	Seguridad	2	2	4 Medio	Rol activo único en el token (ADR-004); filtros por propietario en el dominio	Prueba de API con un usuario docente y estudiante
R-08	Acceso de un estudiante a datos de otro	Privacidad	2	3	**6 Alto**	Filtros por propietario en el dominio de cada módulo (RN-08)	Pruebas de API automatizadas: A pide datos de B → 403
R-09	Robo de token en un computador compartido	Seguridad	2	2	4 Medio	Token de acceso de 15 min; renovación de 8 h con rotación; cookie `HttpOnly`	Revisión de seguridad con OWASP ZAP
R-10	Datos personales tratados por proveedores externos (nube, Cloudflare) sin acuerdo	Legal (Ley 1581 de 2012)	2	3	**6 Alto**	Fuera del alcance técnico	Revisión jurídica de los contratos de tratamiento de datos antes de producción

Id	Riesgo	Tipo	P	I	Nivel	Mitigación en el diseño	Validación
R-11	Los límites entre módulos se erosionan con el tiempo	Mantenibilidad	2	2	4 Medio	Interfaces explícitas y un esquema por módulo (ADR-001)	Pruebas de arquitectura (ArchUnit) en el pipeline
R-12	El respaldo no se puede restaurar cuando se necesita	Recuperación	1	3	3 Medio	WAL continuo, copia diaria, bloqueo de objetos (ADR-003)	Simulacro trimestral midiendo RPO y RTO reales
R-13	La interfaz no cumple accesibilidad ni compatibilidad	Usabilidad	2	2	4 Medio	Requisitos en la matriz	axe-core y Lighthouse CI; pruebas en 4 navegadores con Playwright

Límites de escalabilidad

- **El monolito escala completo.** Si un solo módulo concentra la carga (por ejemplo, Matrículas en semanas pico), hay que replicar toda la aplicación. Es aceptable para 300 usuarios. Si el crecimiento lo exige, el módulo se extrae como servicio con el patrón Strangler Fig, gracias a que ya tiene su propia interfaz y su propio esquema.
- **Una sola base de datos primaria** recibe todas las escrituras. El techo depende del tamaño de la máquina (escalado vertical). Las lecturas pesadas se pueden llevar a una réplica de lectura.
- **Un grupo muy popular** es un punto de contención inevitable: las matrículas a un mismo grupo se procesan en fila. Las de grupos distintos, en paralelo.

Decisiones arquitectónicas que requieren validación

Tabla 23

Decisiones arquitectónicas que requieren validación

Decisión	Hipótesis que hay que comprobar	Prueba propuesta	Criterio de éxito
ADR-001 Monolito modular	Con 2 nodos se atienden 300 usuarios concurrentes	Prueba de carga k6: 70 % consultas, 20 % matrículas, 10 % reportes; de 50 a 500 usuarios	P95 ≤ 3 s con 300 usuarios; registrar el punto de degradación
ADR-002 Bloqueo pesimista	No hay sobrecupo y la espera es aceptable	300 usuarios sobre un grupo con 5 cupos en 2 s	5 matrículas exactas; P95 ≤ 3 s; cero errores 500
ADR-003 Alta disponibilidad	El servicio sobrevive a la caída de un nodo y a la conmutación de la base	Apagar un nodo; forzar la conmutación de la base	Servicio disponible; conmutación ≤ 120 s
ADR-003 Respaldo	RPO ≤ 15 min y RTO ≤ 2 h son alcanzables	Simulacro de desastre: dañar la base en pruebas y restaurar con copia + WAL	Pérdida ≤ 15 min; servicio restaurado en ≤ 2 h
ADR-005 Auditoría transaccional	Ningún cambio queda sin auditar	Prueba de caos con `kill -9` durante matrículas	Matrículas guardadas = eventos de auditoría

Las cinco decisiones principales y su justificación están documentadas en `docs/adr/` (ADR-001 a ADR-005). El enunciado pide al menos tres.

9. Evidencias del uso de Google AI Studio

9.1 Configuración y método

Tabla 24

Configuración y método

Elemento	Valor
Herramienta	Google AI Studio (Playground)
Modelo	Gemini 3.8 Flash (`gemini-3.8-flash`)
Nivel de razonamiento	High
Búsqueda en Google (grounding)	Desactivada, para que el modelo razonara solo con el caso y los requerimientos
Fecha	4 de noviembre de 2026
Conversación original	Enlace a Google AI Studio
Respuestas completas y capturas	docs/ai-studio en el repositorio

Método. El análisis propio de los requerimientos (sección 4) y la decisión preliminar del estilo se hicieron **antes** de consultar a la IA, para tener un criterio contra el cual evaluarla. Luego se enviaron siete prompts en una sola conversación, en el orden del diseño, sin corregir a la IA entre prompts. Cada propuesta de cada respuesta se clasificó como **aceptada**, **modificada** o **descartada**, con una justificación basada en los requerimientos.

9.2 Prompts utilizados

Tabla 25

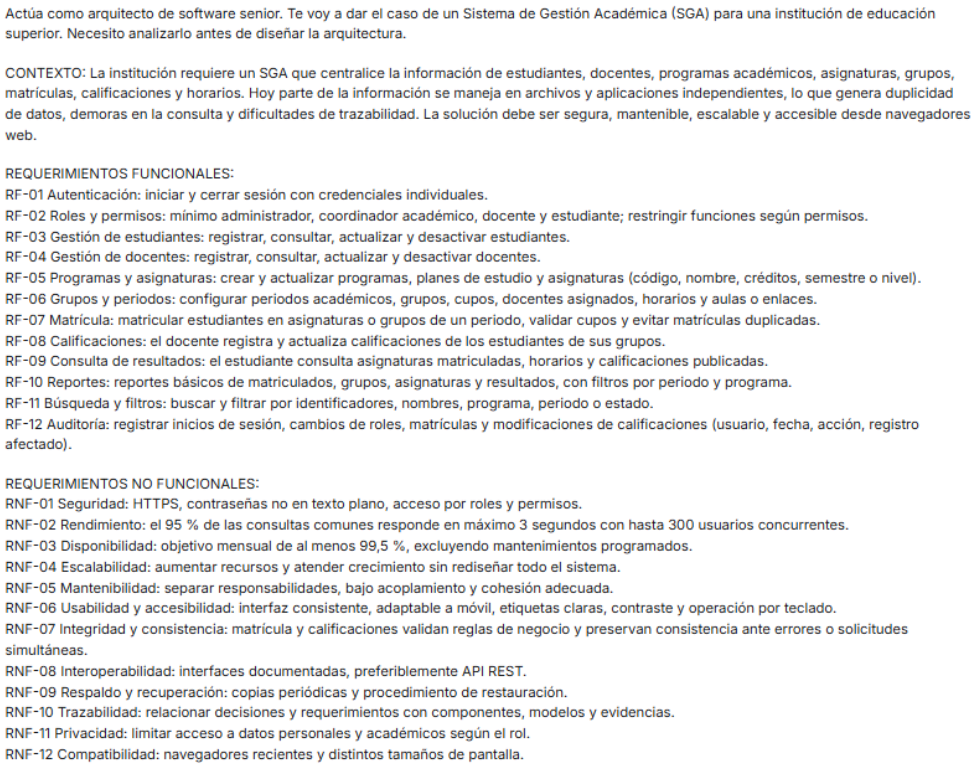
Prompts utilizados

Prompt	Etapa del diseño	Qué se le pidió	Capturas (Anexo B)
P1	Análisis del problema	Actores, supuestos, restricciones, reglas de negocio y clasificación de los RNF, con el caso y los 24 requerimientos completos	P1-01 a P1-06
P2	Estilo arquitectónico	Comparar capas, monolito modular y microservicios frente a los RNF y recomendar uno	P2-01 a P2-04
P3	Componentes	Módulos, responsabilidades, interfaces REST, dependencias y diagrama Mermaid	P3-01 a P3-07
P4	Datos	Entidades, atributos, cardinalidades y restricciones contra matrículas duplicadas	P4-01 a P4-05
P5	Secuencia	Matrícula con validación de cupo, duplicados y dos solicitudes por el último cupo	P5-01 a P5-04
P6	Despliegue	Nodos, canales, seguridad, disponibilidad del 99,5 % y respaldo con presupuesto moderado	P6-01 a P6-05
P7	Autocrítica	Evaluar como auditor externo la propia propuesta: vacíos, fallas, riesgos y validación	P7-01 a P7-06

El texto completo de cada prompt está en el Anexo C.

Figura 8

Prompt P1 enviado a Gemini con el caso y los requerimientos



Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión Académica (SGA) para una institución de educación superior. Necesito analizarlo antes de diseñar la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes, programas académicos, asignaturas, grupos, matrículas, calificaciones y horarios. Hoy parte de la información se maneja en archivos y aplicaciones independientes, lo que genera duplicidad de datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura, mantenible, escalable y accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante; restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas (código, nombre, créditos, semestre o nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados, horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones (usuario, fecha, acción, registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300 usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras, contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y preservan consistencia ante errores o solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

Nota. Captura de la conversación en Google AI Studio. Fuente: Google AI Studio (2026).

9.3 Resultado de la evaluación

Tabla 26

Resultado de la evaluación

Prompt	Aceptadas	Modificadas	Descartadas	Observación
P1 Análisis	11	6	1	Aportó tres reglas que no se tenían: fechas límite, cierre de notas y cruce de horarios
P2 Estilo	5	5	—	Recomendó el mismo estilo; se corrigió que capas y módulos se combinan
P3 Componentes	5	9	1	Auditoría asíncrona que podía perder eventos; bloqueo sobre tablas de otro módulo
P4 Datos	6	7	1	Faltaban planes de estudio, estado de publicación y fechas del periodo; porcentaje mal normalizado
P5 Secuencia	2	7	—	Aportó el orden de bloqueos contra interbloqueos; validaba dentro del bloqueo
P6 Despliegue	10	4	1	La más útil: base de datos en alta disponibilidad, respaldo continuo, Cloudflare

Prompt	Aceptadas	Modificadas	Descartadas	Observación
P7 Autocrítica	8	2	2	Tres de sus críticas ya estaban corregidas en el diseño del autor
Total	**47**	**40**	**6**	**93 propuestas evaluadas**

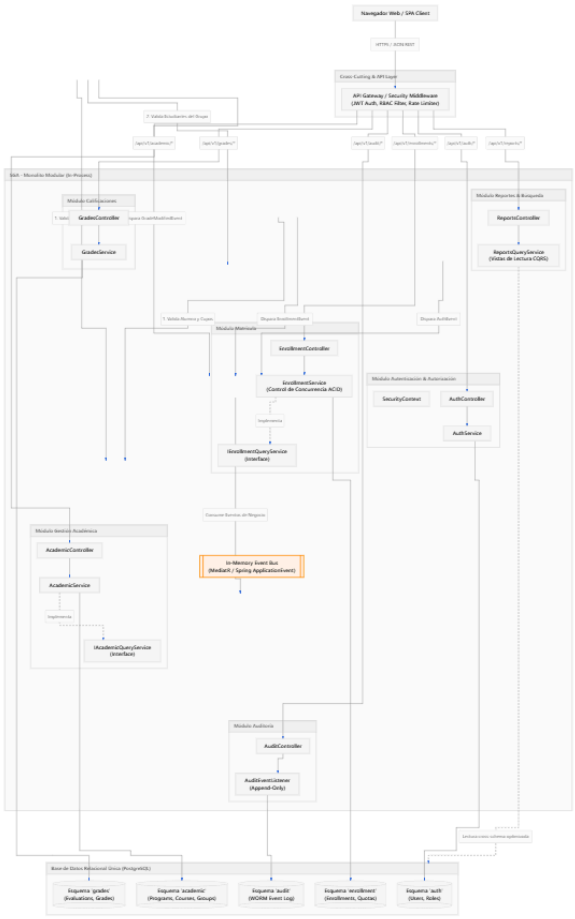
La evaluación detallada de las 93 propuestas, una por fila y con su justificación, está en el Anexo A.

9.4 Ajustes más importantes realizados a las propuestas de la IA

1. **Auditoría (P3, P5).** La IA propuso registrar la auditoría con un bus de eventos asíncrono después del COMMIT. Si el proceso se cae entre ambos pasos, el cambio queda sin auditar, lo que viola RF-12. Se reemplazó por una escritura en la misma transacción (ADR-005). En su autocrítica (P7), la propia IA reconoció este riesgo.
2. **Aislamiento entre módulos (P3, P4, P5).** La IA defendió que ningún módulo toca las tablas de otro, pero luego bloqueó la tabla Grupo desde Matrículas. Se resolvió con la entidad CupoGrupo, propiedad de Matrículas (ADR-002).
3. **Sobredimensionamiento.** Redis se propuso cuatro veces, además de almacenamiento WORM, colas de matrícula, servicio de notificaciones y ABAC. Se descartaron o simplificaron por no tener un requerimiento que los justifique (S-08).
4. **Vacíos frente al enunciado (P4).** Se agregaron planes de estudio (RF-05), estado de publicación de notas (RF-09) y fechas del periodo (RN-11), y se normalizó el porcentaje de las evaluaciones.
5. **Mejoras aceptadas.** De la IA se tomaron el cierre de notas, el cruce de horarios, el orden de bloqueos, la recuperación a un punto en el tiempo (RPO 15 min), la capacidad del aula, el rol activo y el plan de pruebas de validación.

Figura 9

Diagrama de componentes generado por la IA (P3)



Nota. El diagrama de la IA resultó ilegible: líneas cruzadas, un recuadro vacío y nombres en dos idiomas. Se redibujó como la Figura 2. Fuente: Google AI Studio (2026).

9.5 Conclusiones sobre el uso de la IA

1. **Acertó en lo estructural.** Recomendó el mismo estilo que el equipo (monolito modular), los mismos módulos y el bloqueo pesimista. Su comparación de estilos y su plan de pruebas fueron de buena calidad.
2. **Aportó ideas que el equipo no tenía:** el cierre de calificaciones, el cruce de horarios, el orden de los bloqueos para evitar interbloqueos, la recuperación a un punto en el tiempo, la capacidad del aula y el contexto de rol activo.
3. **Tendió a sobredimensionar:** Redis (cuatro veces), WORM, colas, notificaciones y ABAC, que agregan costo y componentes sin un requerimiento que los pida.
4. **Fue inconsistente consigo misma:** defendió el aislamiento entre módulos y después bloqueó tablas de otro módulo; dijo "asíncrona" y "sincrónica" para la misma auditoría. Repitió errores ya corregidos en prompts siguientes. **No basta con corregirla una vez: cada respuesta se debe revisar contra los requerimientos.**

5. **Los diagramas que generó fueron ilegibles** en tres de cuatro casos. Los modelos finales se redibujaron.

10. Video de presentación y repositorio

Tabla 27

Video de presentación y repositorio

Entregable	Enlace
Video o *pitch* en YouTube	**[Enlace pendiente: se agrega cuando el video esté publicado en YouTube]**
Repositorio del proyecto (GitHub)	https://github.com/juanvs1985/sga-arquitectura

El repositorio contiene el análisis, los seis modelos con sus archivos fuente (Mermaid y SVG) y exportados (PNG), las cinco decisiones arquitectónicas, el análisis de riesgos y todas las evidencias de Google AI Studio (respuestas completas, capturas y evaluación).

11. Conclusiones y recomendaciones

11.1 Conclusiones

1. **Un monolito modular en capas cubre los 24 requerimientos** con menos complejidad y costo que una arquitectura de microservicios. La decisión se sostiene en los atributos de calidad del caso, en especial la consistencia de la matrícula (RNF-07), y no en preferencias tecnológicas.
2. **La matrícula es el corazón técnico del sistema.** Combina reglas de negocio, concurrencia y auditoría. Resolverla con una transacción, un bloqueo de fila y restricciones en la base de datos garantiza que no haya sobrecupo aunque falle la lógica de la aplicación.
3. **La disponibilidad del 99,5 % se logra eliminando los puntos únicos de falla internos** (nodos, balanceador y base de datos redundantes). Los riesgos residuales (Cloudflare y la región del proveedor) quedan identificados y aceptados de forma explícita.
4. **La trazabilidad es completa en el diseño:** cada requerimiento tiene componente, modelo, decisión y método de verificación. Los menos sustentados (accesibilidad y compatibilidad) dependen de la implementación de la interfaz.
5. **La IA aceleró el diseño, pero no lo reemplazó.** Acertó en lo estructural y aportó ideas valiosas, pero también sobredimensionó, se contradijo y repitió errores ya corregidos. Solo la revisión sistemática contra los requerimientos permitió aprovechar lo útil y descartar lo incorrecto.

11.2 Recomendaciones

1. **Antes de implementar,** ejecutar las pruebas de validación de la sección 8: carga con 300 usuarios, concurrencia sobre el último cupo, caída de un nodo y simulacro de restauración.
2. **Validar con la institución** los supuestos (en especial quién matricula y los valores de RPO y RTO) y revisar los contratos de tratamiento de datos con los proveedores (Ley 1581 de 2012).
3. **Construir primero los módulos de mayor riesgo** (Identidad y acceso, y Matrículas) e incluir desde el inicio las pruebas de arquitectura que protegen los límites entre módulos.
4. **Si se habilita la automatrícula,** repetir la prueba de contención extrema y reconsiderar la estrategia de bloqueo antes de abrirla.
5. **Al usar IA en diseño de software,** hacer un análisis propio antes de consultarla, pedirle una autocrítica al final y registrar qué se acepta y qué no, con su justificación.

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.° 48.587.

Google. (2026). *Gemini 3.8 Flash* [Modelo de lenguaje de gran tamaño]. Google AI Studio. https://aistudio.google.com

International Organization for Standardization. (2011). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models* (ISO/IEC 25010:2011).

Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, *12*(6), 42–50. https://doi.org/10.1109/52.469759

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation: Explicit locking*. https://www.postgresql.org/docs/current/explicit-locking.html

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico* (9.ª ed.). McGraw-Hill Education.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

Anexo A. Evaluación detallada de las respuestas de la IA

P1 · Análisis del problema

Tabla 28

P1 · Análisis del problema

#	Propuesta de la IA	Decisión	Justificación
1.1	Cuatro actores principales: estudiante, docente, coordinador académico, administrador.	Aceptado	Coincide con RF-02 y con el análisis propio.
1.2	El **estudiante realiza su propia matrícula**.	Modificado	RF-07 no dice quién matricula. Mantenemos al coordinador como responsable (supuesto S-02) para reducir el riesgo de concurrencia masiva en esta primera versión. La autoinscripción queda como evolución: la API de matrícula es la misma y solo cambia el permiso.
1.3	**Servicio de auditoría** como actor secundario.	Modificado	La auditoría es un módulo interno del sistema (RF-12), no un actor externo. Se modela como componente, no en el diagrama de casos de uso.
1.4	Servicio de copias de seguridad como actor secundario.	Aceptado	Coincide con el análisis propio (RNF-09).
1.5	**Servicio de notificaciones** (correo/SMS).	Descartado	Ningún RF lo pide. Agregarlo amplía el alcance sin requisito que lo justifique. Se menciona como recomendación futura.
1.6	Baja lógica (soft delete) al desactivar estudiantes y docentes.	Aceptado	Igual al supuesto S-03; preserva el historial y la integridad referencial.
1.7	**Ventanas de tiempo** para matrícula y registro de notas.	Aceptado	Amplía nuestra regla RN-03 (periodo abierto). Se agrega como RN-11.
1.8	Calificaciones ponderadas en escala 0,0–5,0 con nota aprobatoria configurable.	Aceptado	Coincide con RN-06.
1.9	**Validar cruces de horario** de estudiantes y docentes.	Aceptado	Buena regla que el análisis propio no tenía. Se agrega como RN-13.
1.10	Control de concurrencia en la matrícula: optimista, pesimista **o colas**.	Modificado	Se elige bloqueo pesimista de la fila del grupo dentro de una transacción, más una restricción única en la base. Las colas se descartan: con 300 usuarios concurrentes agregan complejidad sin beneficio (ver ADR-002).
1.11	Desacoplar el proveedor de identidad para un SSO futuro (OAuth2, SAML, Google o Microsoft).	Modificado	Se acepta como punto de extensión del módulo Identidad y acceso, pero queda fuera del alcance actual: RF-01 pide credenciales propias.
1.12	Restricciones técnicas: navegador, API REST con OpenAPI, TLS, hash Argon2id o bcrypt, 3,6 h de caída al mes.	Aceptado	Coinciden con R-01 a R-05.
1.13	Caché con **Redis** para catálogos y **CDN** para estáticos.	Pendiente (se decide en P2)	Hay que validar si se justifican para 300 usuarios y presupuesto moderado.

#	Propuesta de la IA	Decisión	Justificación
1.14	Auditoría **WORM** (almacenamiento de una sola escritura).	Modificado	WORM es costoso para el presupuesto supuesto (S-08). Se cumple el mismo objetivo con una tabla de auditoría de solo inserción: la cuenta de la aplicación no tiene permisos de UPDATE ni DELETE sobre ella.
1.15	Cierre y **congelamiento de calificaciones**: después del cierre, el docente no edita; cualquier cambio requiere autorización con auditoría.	Aceptado	Regla valiosa que el análisis propio no tenía. Se agrega como RN-12.
1.16	Auditoría inmutable (solo adición).	Aceptado	Se agrega como RN-14, implementada como se indica en 1.14.
1.17	Control de acceso **ABAC/RBAC**.	Modificado	Se usa RBAC (RF-02) más filtros de propiedad en el dominio ("solo lo mío", "solo mis grupos"). ABAC completo no se justifica en este alcance.
1.18	Clasificación de los RNF según ISO/IEC 25010.	Aceptado	Coincide con la tabla 1.7 del análisis. Se adopta la subcaracterística "Recuperabilidad" para RNF-09.
1.19	IDs de correlación en las trazas (RNF-10).	Aceptado	Mejora la trazabilidad técnica de cada petición entre los módulos y la auditoría.

Balance P1: 11 aceptadas, 6 modificadas, 1 descartada, 1 pendiente. La IA aportó tres reglas de negocio que el análisis propio no tenía: ventanas de tiempo, cierre de calificaciones y cruces de horario. En cambio, tendió a sobredimensionar la solución (WORM, colas, notificaciones, ABAC) frente al tamaño del problema.

P2 · Estilo arquitectónico

Tabla 29

P2 · Estilo arquitectónico

#	Propuesta de la IA	Decisión	Justificación
2.1	Recomendar **monolito modular** con una base de datos relacional y API REST.	Aceptado	Coincide con la decisión que el equipo ya había tomado antes de consultar a la IA. Se documenta en ADR-001 con argumentos propios: tamaño del sistema, 300 usuarios concurrentes, consistencia de la matrícula y un solo desarrollador.
2.2	Presentar "capas" y "monolito modular" como **alternativas excluyentes**, y advertir que las capas terminan en una "gran bola de lodo".	Modificado	No son excluyentes. La propuesta final combina ambos: módulos verticales por contexto de negocio y, dentro de cada módulo, capas de presentación (controlador), dominio y repositorio. El riesgo de la gran bola de lodo viene de no tener límites entre módulos, no de usar capas.
2.3	Descartar microservicios por complejidad operativa, latencia de red y consistencia eventual (Sagas) en la matrícula.	Aceptado	Argumento correcto y verificable. Además, el equipo es de una persona, lo que refuerza el descarte.
2.4	Bloqueo pesimista (`SELECT … FOR UPDATE`) **o** control optimista para los cupos.	Modificado	Se elige el bloqueo pesimista de la fila del grupo más una restricción única. Con contención alta por el último cupo, el control optimista produce muchos reintentos fallidos (ver ADR-002).

#	Propuesta de la IA	Decisión	Justificación
2.5	**Partición lógica de esquemas**: una base de datos con un esquema por módulo.	Aceptado	Refuerza el bajo acoplamiento: cada módulo es dueño de sus tablas y los demás solo acceden por su interfaz pública. Facilita extraer un módulo en el futuro.
2.6	2 o 3 réplicas en contenedores Docker detrás de un balanceador para el 99,5 %.	Aceptado	Se fijan 2 réplicas como mínimo. Requisito previo: la API debe ser sin estado (sesión en token, no en memoria).
2.7	Auditoría mediante **interceptores (middleware)**.	Modificado	Un middleware no conoce el valor anterior y el nuevo de una nota. Se usa middleware para los inicios de sesión, y para matrículas, notas y roles el módulo de dominio escribe el registro de auditoría **dentro de la misma transacción** del cambio: si el cambio se guarda, la auditoría también.
2.8	Afirmaciones como rendimiento "excelente" o "supera con holgura los 3 s".	Modificado	Son afirmaciones no verificadas. Se convierten en hipótesis que se comprobarán con una prueba de carga de 300 usuarios virtuales (ver matriz de trazabilidad, RNF-02).
2.9	Si un módulo crece, extraerlo a microservicio en el futuro.	Aceptado	Se documenta como camino de evolución con el patrón Strangler Fig.
1.13	(Pendiente del P1) Caché con **Redis** y **CDN**.	Redis: Descartado · CDN: Modificado	Redis: con 300 usuarios y sesión en token no hay estado compartido que guardar; los catálogos (programas, asignaturas) se cachean en memoria de cada réplica. Se reconsidera si la prueba de carga no cumple RNF-02. CDN: la interfaz web estática se publica en un hosting que ya incluye distribución por CDN, sin costo ni componente adicional.

Balance P2: 5 aceptadas, 5 modificadas (incluido el pendiente del P1). La recomendación de la IA coincide con la del equipo, pero la propuesta final corrige tres puntos: capas y módulos se combinan, la auditoría va en la misma transacción y el rendimiento se trata como una hipótesis por verificar.

P3 · Componentes y vista lógica

Tabla 30

P3 · Componentes y vista lógica

#	Propuesta de la IA	Decisión	Justificación
3.1	Seis módulos: autenticación y autorización, gestión académica, matrícula, calificaciones, reportes y búsqueda, auditoría.	Aceptado	Coinciden con los seis módulos de ADR-001. En el modelo final se nombran en español para mantener la consistencia con las entidades del enunciado.
3.2	Interfaces internas `IAcademicQueryService` e `IEnrollmentQueryService`, sin consultas SQL de un módulo a las tablas de otro.	Aceptado	Aplica el principio de inversión de dependencias y protege los límites entre módulos (RNF-05). Se renombran `IConsultaAcademica` e `IConsultaMatriculas`.

#	Propuesta de la IA	Decisión	Justificación
3.3	Dependencias: Matrícula → Gestión académica; Calificaciones → Gestión académica y Matrícula.	Aceptado	No hay ciclos entre módulos. Se verifica en el diagrama propio.
3.4	**"API Gateway"** como capa de entrada.	Modificado	En un monolito no hace falta un producto de API Gateway aparte. La seguridad (JWT, roles, límite de peticiones, manejo de errores) es un *pipeline* de middleware dentro de la misma aplicación, y el HTTPS termina en el balanceador (vista de despliegue).
3.5	Auditoría por un **bus de eventos en memoria asíncrono**.	Modificado	Error de diseño. Si el proceso se cae después de guardar la nota pero antes de procesar el evento, el cambio queda sin auditar, lo que viola RF-12. Además, la respuesta se contradice: en un punto dice "asíncrona" y en otro "sincrónicamente". Se reemplaza por la interfaz `IRegistroAuditoria`, que el módulo llama **dentro de la misma transacción** del cambio (ADR-005).
3.6	La matrícula abre la transacción en el esquema `enrollment`, pero bloquea la **fila del grupo**, que vive en el esquema académico.	Modificado	Inconsistencia: Matrícula estaría escribiendo en tablas de Gestión académica y rompería el aislamiento que la misma IA defiende en 3.2. Se resuelve con lo que la IA ya insinuaba en su esquema (`Quotas`): Matrícula es dueña de la tabla `cupo_grupo` (cupo máximo y ocupados) y el bloqueo se hace sobre esa fila (ADR-002).
3.7	Cerrar sesión con **lista negra de tokens en Redis**.	Modificado	Redis se descartó en P2. Se usa un token de acceso de vida corta (15 min) más un token de renovación guardado en el esquema de identidad, que se revoca al cerrar sesión (ADR-004).
3.8	Endpoints REST versionados `/api/v1/...` por módulo.	Aceptado	Cumple RNF-08. Se documentarán con OpenAPI.
3.9	Faltan endpoints para **publicar** calificaciones y **cerrar** el periodo de notas.	Modificado (agregado)	Sin ellos no se cumplen RF-09 (el estudiante solo ve notas publicadas) ni RN-12 (cierre). Se agregan `POST /calificaciones/grupos/{id}/publicar` y `POST /academico/periodos/{id}/cerrar-notas`.
3.10	Prerrequisitos de asignaturas y datos de *tenant*.	Descartado	Ningún RF pide prerrequisitos ni multiinstitución. Se dejan como evolución futura.
3.11	Reportes con lectura directa de varias tablas mediante vistas SQL (CQRS ligero) y **réplicas de lectura**.	Modificado	Se aceptan las vistas de solo lectura con un usuario de base de datos sin permisos de escritura. La réplica de lectura solo se agrega si la prueba de carga muestra que los reportes afectan a RNF-02.
3.12	Búsqueda global `GET /search?q=` en el módulo de reportes.	Modificado	RF-11 pide buscar "según el módulo". La búsqueda va en los listados de cada módulo (`?q=&periodo=&programa=&estado=`) y aplica los filtros del rol (RNF-11).
3.13	Exportar reportes a PDF y Excel.	Modificado	RF-10 pide reportes básicos. Exportar a CSV queda como "debería" y a PDF como "podría".
3.14	Justificación obligatoria al modificar una nota, guardada en la auditoría.	Aceptado	Refuerza RN-12 y RF-12.

#	Propuesta de la IA	Decisión	Justificación
3.15	Diagrama Mermaid generado.	Modificado	El código es válido, pero el diagrama resultante es **ilegible** (ver `P3-diagrama-ia.png`): muchas líneas cruzadas, el bus de eventos aparece como un recuadro vacío y mezcla nombres en inglés y español. El modelo final se redibuja en dos diagramas más simples (módulos y estructura interna de un módulo).

Balance P3: 5 aceptadas, 9 modificadas, 1 descartada. Fue la respuesta con más correcciones. La IA describió bien los módulos y sus interfaces, pero su propuesta de auditoría asíncrona y el bloqueo sobre tablas de otro módulo contradecían requisitos y principios que ella misma había defendido en P2.

P4 · Modelo de datos conceptual

Tabla 31

P4 · Modelo de datos conceptual

#	Propuesta de la IA	Decisión	Justificación
4.1	Las diez entidades del enunciado más `Horario`, `UsuarioRol` y `Auditoria`, con atributos y claves.	Aceptado	Cubre las entidades exigidas en 6.4. El diagrama ER generado es legible (`P4-diagrama-ia.png`).
4.2	Relación **M:N entre Usuario y Rol** (`UsuarioRol`).	Aceptado	Mejor que nuestro supuesto S-05 (un rol por usuario): resuelve con naturalidad al docente que además coordina. Se actualiza S-05.
4.3	Estudiante y Docente como extensión 1:0..1 de Usuario.	Aceptado	Separa las credenciales (Identidad) de los datos académicos (Gestión académica).
4.4	Matrícula en **cabecera y detalle** (`Matricula` por periodo y `DetalleMatricula` por grupo).	Aceptado	Representa bien "matricular en asignaturas o grupos de un periodo" (RF-07).
4.5	`id_asignatura` **desnormalizado** en `DetalleMatricula` para la restricción única.	Modificado	Sin control, un detalle podría quedar con un grupo de una asignatura y un `id_asignatura` de otra. Se agrega una clave foránea compuesta `(id_grupo, id_asignatura)` → `Grupo(id_grupo, id_asignatura)` que obliga a que coincidan.
4.6	`cupo_ocupado` dentro de **Grupo** y bloqueo `FOR UPDATE` sobre Grupo.	Modificado	Repite el error del P3: Matrícula escribiría en tablas de Gestión académica. Grupo conserva `cupo_maximo` y se agrega la entidad **CupoGrupo** (1:1 con Grupo, del módulo Matrículas) con `ocupados` (ADR-002).
4.7	`porcentaje` del corte guardado en **cada calificación**.	Modificado	Error de normalización: el 30 % del "Parcial 1" se repetiría en la nota de cada estudiante y podría quedar distinto entre ellos. Se crea la entidad **Evaluacion** (corte del grupo con nombre y porcentaje); `Calificacion` queda como la nota de un estudiante en una evaluación.
4.8	Calificación **sin estado de publicación**.	Modificado (agregado)	RF-09 exige que el estudiante vea solo notas publicadas. Se agrega `estado` (Borrador o Publicada) en Calificación.

#	Propuesta de la IA	Decisión	Justificación
4.9	Asignatura cuelga directamente de Programa; **no hay plan de estudios**.	Modificado (agregado)	RF-05 pide gestionar planes de estudio. Se agrega **PlanEstudio** (Programa 1:N PlanEstudio 1:N Asignatura).
4.10	PeriodoAcademico solo con fecha de inicio, fecha de fin y estado.	Modificado	Faltan las fechas de matrícula y de registro de notas, y el cierre (RN-11, RN-12). Se agregan `inicio_matricula`, `fin_matricula`, `limite_notas` y `notas_cerradas`.
4.11	Restricciones únicas `(id_estudiante, id_periodo)`, `(id_matricula, id_asignatura)`, `(id_matricula, id_grupo)` y `CHECK (ocupados <= cupo_maximo)`.	Aceptado	Son la segunda barrera de ADR-002 en la base de datos. El CHECK pasa a la entidad CupoGrupo.
4.12	Cruce de horarios con `EXCLUDE USING GIST`.	Descartado	Esa restricción compara filas de una misma tabla; el cruce de un estudiante involucra Horario, Grupo y DetalleMatricula. Se valida en el servicio de dominio dentro de la transacción (RN-13).
4.13	Auditoría con valores anteriores y nuevos en JSONB.	Aceptado	Cumple RF-12 y ADR-005. Se descarta otra vez la etiqueta WORM (ver 1.14).
4.14	Falta la entidad para los **tokens de renovación**.	Modificado (agregado)	ADR-004 la necesita para poder cerrar sesión. Se agrega **Sesion** en el esquema de identidad.

Balance P4: 6 aceptadas, 7 modificadas, 1 descartada. El modelo de la IA es sólido como base, pero le faltaban tres elementos que el enunciado pide (planes de estudio, notas publicadas y fechas del periodo) y tenía un problema de normalización en los porcentajes.

P5 · Secuencia de la matrícula

Tabla 32

P5 · Secuencia de la matrícula

#	Propuesta de la IA	Decisión	Justificación
5.1	Escenario de dos solicitudes simultáneas por el último cupo: Tx1 bloquea, Tx2 espera, Tx1 confirma, Tx2 lee 30/30, revierte y recibe **409 Conflict**.	Aceptado	Es exactamente el comportamiento que exige RNF-07 y el escenario de prueba de ADR-002.
5.2	**Ordenar los `id_grupo` de forma ascendente** antes de bloquear cuando se matriculan varios grupos a la vez.	Aceptado	Aporte valioso que el equipo no había considerado: evita interbloqueos (*deadlocks*) entre dos matrículas con grupos en común. Se agrega a ADR-002.
5.3	Bloqueo sobre la tabla **Grupo**.	Modificado	Mismo error que en P3 y P4. El bloqueo va sobre CupoGrupo (ADR-002).

#	Propuesta de la IA	Decisión	Justificación
5.4	Valida el estado del estudiante y los duplicados **después** de tomar el bloqueo.	Modificado	Así el bloqueo se retiene más tiempo del necesario. Las validaciones que no dependen del cupo (estudiante activo, periodo abierto, cruce de horario) se hacen **antes** del bloqueo. Dentro del bloqueo solo quedan el cupo, el duplicado y la escritura.
5.5	Auditoría **asíncrona** después del COMMIT "para no frenar la respuesta".	Modificado	Tercera vez que la IA lo propone. Se mantiene ADR-005: la auditoría va dentro de la transacción. Una inserción adicional cuesta milisegundos y garantiza que ninguna matrícula quede sin registro.
5.6	Bloqueo retenido "≤ 15 ms".	Modificado	Es una cifra no verificada. Se trata como hipótesis y se mide en la prueba de concurrencia de ADR-002.
5.7	No define qué pasa si la espera del bloqueo se alarga.	Modificado (agregado)	Se agrega un tiempo máximo de espera (`lock_timeout` de 5 s) con respuesta "intente de nuevo" (503), para no agotar las conexiones del pool.
5.8	El actor que matricula es el **estudiante**.	Modificado	Según el supuesto S-02, en esta versión matricula el coordinador. El flujo es el mismo; solo cambia el rol autorizado.
5.9	Participante "API Gateway / Auth".	Modificado	Se reemplaza por el pipeline de seguridad dentro del monolito (ver 3.4).

Balance P5: 2 aceptadas, 7 modificadas. La IA resolvió bien el problema central de concurrencia y aportó la regla de ordenar los bloqueos. Pero insistió en la auditoría asíncrona y en bloquear la tabla de otro módulo, aunque ya se habían corregido.

P6 · Despliegue, seguridad y respaldo

Tabla 33

P6 · Despliegue, seguridad y respaldo

#	Propuesta de la IA	Decisión	Justificación
6.1	**Cloudflare** (plan gratuito) delante del sistema: TLS, mitigación de DDoS, WAF básico y CDN.	Aceptado	Cubre RNF-01 y la CDN que quedó pendiente en P2 sin costo adicional. También permite el límite de peticiones (*rate limiting*) en el borde.
6.2	Interfaz web compilada en **almacenamiento de objetos + CDN**.	Aceptado	Los servidores de la API no gastan CPU en archivos estáticos.
6.3	Balanceador (Nginx o el del proveedor) con *health checks* y **2 instancias** de la API (2 vCPU, 4 GB) en red privada.	Aceptado	Coincide con ADR-001 y ADR-004. El tamaño de las máquinas es una estimación que se confirma con la prueba de carga (RNF-02).
6.4	**Redis** para lista negra de tokens, límite de peticiones y caché.	Descartado	Cuarta vez que la IA lo propone. Ya se resolvió sin Redis: los tokens en ADR-004, el límite de peticiones en Cloudflare y la caché de catálogos en memoria (P2). Agregarlo sumaría un componente y un punto de falla sin necesidad.
6.5	**PostgreSQL administrado con réplica en espera** y conmutación automática (60 a 120 s).	Aceptado	Elimina el punto único de falla que ADR-001 había dejado abierto. Se exige que la réplica esté en otra zona de disponibilidad. Se documenta en ADR-003.

#	Propuesta de la IA	Decisión	Justificación
6.6	Base de datos en **subred privada** sin IP pública; el puerto 5432 solo acepta tráfico de los nodos de la API; conexiones con TLS.	Aceptado	Defensa en profundidad para RNF-01 y RNF-11.
6.7	Secretos (llaves JWT, credenciales) fuera del repositorio, inyectados por un **gestor de secretos**.	Aceptado	Cumple además la indicación del enunciado de no publicar credenciales en el repositorio.
6.8	Token de renovación de **7 días** con rotación en cada uso.	Modificado	Se acepta la rotación (mejora ADR-004). Se mantiene una vigencia de 8 horas: en una institución educativa muchos usuarios entran desde computadores compartidos (salas, bibliotecas), y 7 días es demasiado.
6.9	**RPO ≤ 15 min** y **RTO ≤ 2 h** con respaldo diario completo más archivado continuo del WAL (recuperación a un punto en el tiempo).	Aceptado	Mejora nuestra propuesta inicial (RPO 24 h, RTO 4 h) con bajo costo, porque los servicios administrados ya incluyen PITR. Se actualiza RNF-09 en el análisis.
6.10	Respaldos cifrados (AES-256) en almacenamiento externo, regla 3-2-1, **Object Lock** con retención de 30 días.	Aceptado	Protege contra borrado accidental y *ransomware*. A diferencia de la auditoría (1.14), aquí el bloqueo de objetos es una función barata del almacenamiento y sí se justifica.
6.11	Restauración con un script probado **cada trimestre** en un entorno de pruebas.	Aceptado	Es la evidencia de cumplimiento de RNF-09 en la matriz de trazabilidad.
6.12	Despliegues **rolling update** sin interrupción.	Aceptado	Evita que las actualizaciones consuman el margen de 3,6 h de RNF-03.
6.13	No menciona **monitoreo ni alertas**.	Modificado (agregado)	Sin monitoreo no se puede medir el 99,5 % ni reaccionar a tiempo. Se agrega un monitor externo de disponibilidad con alertas, y registros centralizados con el identificador de correlación.
6.14	No considera que Cloudflare y el proveedor de nube **procesan datos personales** de estudiantes.	Modificado (agregado)	En Colombia aplica la Ley 1581 de 2012 (habeas data). Se registra como riesgo que requiere validar los contratos de tratamiento de datos con los proveedores.
6.15	Diagrama de despliegue generado.	Modificado	Otra vez ilegible (`P6-diagrama-ia.png`): textos que no se leen y flechas sueltas. Se redibuja.

Balance P6: 10 aceptadas, 4 modificadas, 1 descartada. Fue la respuesta más útil de las seis: elevó el nivel de la propuesta en disponibilidad y respaldo.

P7 · Revisión crítica de la propia propuesta

La IA criticó **su propia** propuesta de esta conversación. Varias de sus críticas más graves apuntan a elementos que el equipo ya había corregido en las evaluaciones anteriores. Eso confirma esas correcciones.

Tabla 34

P7 · Revisión crítica de la propia propuesta

#	Crítica de la IA	Decisión	Justificación
7.1	**Redis único** es punto único de falla (límite de peticiones, lista negra, caché).	Ya resuelto	Redis se descartó en P2, P3 y P6. El diseño final no lo usa.
7.2	La auditoría por **bus de eventos en memoria** se pierde si el proceso cae después del COMMIT; sugiere el patrón *Outbox*.	Ya resuelto	ADR-005 escribe la auditoría en la misma transacción del cambio. Es más simple que *Outbox* y no deja eventos pendientes. La prueba de caos que propone la IA (`kill -9` durante matrículas) se adopta para **demostrar** que no hay discrepancias.
7.3	Bloqueo sobre la **tabla Grupo**.	Ya resuelto	ADR-002 bloquea `cupo_grupo`, que pertenece al módulo Matrículas.
7.4	**Balanceador único** como punto único de falla.	Aceptado	Crítica válida. Se usa el balanceador administrado del proveedor, que es redundante por diseño, en lugar de un Nginx propio en una sola máquina. Se actualiza ADR-003.
7.5	**Agotamiento del pool de conexiones** de PostgreSQL con dos nodos.	Aceptado	Se fija un pool máximo por nodo (20 conexiones) y se usa el *pooler* que ofrece el servicio administrado (tipo PgBouncer). Se actualiza ADR-003.
7.6	**Contención extrema** si 250 usuarios piden el mismo grupo al mismo tiempo: la cola de bloqueos supera los 3 s.	Modificado	El riesgo es real si se habilita la automatrícula. Hoy matricula el coordinador (S-02), lo que reduce mucho la concurrencia. Se agrega a ADR-002 una **lectura previa sin bloqueo**: si el grupo ya está lleno, se rechaza de inmediato sin hacer cola. Se valida con la prueba de 300 usuarios sobre 5 cupos que propone la IA.
7.7	Los **reportes pesados** degradan el pool transaccional en el cierre de notas.	Aceptado	Los reportes usan un pool de conexiones separado y pequeño, con tiempo máximo por consulta. La réplica de lectura se activa si la prueba de carga lo exige (ver 3.11).
7.8	La **capacidad del aula** no se considera: un grupo de 40 podría quedar en un aula de 25.	Aceptado	Buena observación sobre RF-06. Se agrega la entidad **Aula** con su capacidad y la regla RN-15: en grupos presenciales, el cupo no puede superar la capacidad del aula.
7.9	Un **superusuario de la base de datos** puede alterar la auditoría.	Aceptado	Riesgo residual real. Se mitiga con una exportación diaria de los eventos de auditoría al almacenamiento con bloqueo de objetos (el mismo de los respaldos, ADR-003) y con el registro de actividad del propio servicio administrado. Así, una alteración posterior se puede detectar comparando.
7.10	**Usuarios con varios roles** sin un contexto de sesión activo.	Aceptado	Crítica válida que surge de aceptar el M:N en P4. El usuario elige el perfil con el que entra y el token lleva un único **rol activo**. Los permisos se evalúan con ese rol. Se actualiza ADR-004.
7.11	RNF-06 sin verificación automática.	Aceptado	Se incluyen axe-core y Lighthouse CI en el pipeline de integración continua (matriz de trazabilidad, RNF-06).
7.12	RNF-10 sin estándar de ADR ni versionado de la API.	Modificado	La IA no conoce los ADR del equipo (ADR-001 a ADR-005 ya existen). Se acepta lo que sí faltaba: versionado semántico de la especificación OpenAPI y commits que citan el requerimiento (por ejemplo, `RF-07: …`).

#	Crítica de la IA	Decisión	Justificación
7.13	Faltan **prerrequisitos** de asignaturas (RF-05).	Descartado	RF-05 no los pide (pide código, nombre, créditos y semestre). Ya se había descartado en 3.10. Queda como evolución futura.
7.14	Calificación **a ciegas** (*blind grading*).	Descartado	No lo pide ningún requerimiento, y el docente necesita saber a quién califica. La privacidad de las notas ya la cubre RNF-11 con filtros por rol y propietario.
7.15	Plan de validación: prueba de concurrencia extrema, prueba de caos, simulacro de desastre y prueba de carga con mezcla 70 % lecturas, 20 % matrículas y 10 % reportes, de 50 a 500 usuarios.	Aceptado	Plan concreto y medible. Se incorpora al análisis de riesgos y a la matriz de trazabilidad.

Balance P7: 8 aceptadas, 2 modificadas, 2 descartadas y 3 que ya estaban resueltas.

Anexo B. Capturas de la conversación en Google AI Studio

Capturas en el orden de la conversación. La primera captura de cada prompt muestra el texto enviado; las siguientes, la respuesta. La captura P5-04 muestra al final el envío del prompt P6.

P1 · Análisis del problema

Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión Académica (SGA) para una institución de educación superior. Necesito analizarlo antes de diseñar la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes, programas académicos, asignaturas, grupos, matrículas, calificaciones y horarios. Hoy parte de la información se maneja en archivos y aplicaciones independientes, lo que genera duplicidad de datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura, mantenible, escalable y accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante; restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas (código, nombre, créditos, semestre o nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados, horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones (usuario, fecha, acción, registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300 usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras, contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y preservan consistencia ante errores o solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

P1-01

Como arquitecto de software senior, a continuación presento el análisis previo de requisitos del Sistema de Gestión Académica (SGA), estructurado formalmente para servir de base al diseño de la arquitectura.

---

1. Identificación de Actores

Actores Principales (Inician casos de uso para cumplir un objetivo de negocio)

- **Estudiante:** Consulta oferta académica, realiza matrícula de asignaturas, revisa horarios asignados y consulta el historial/calificaciones publicadas.
- **Docente:** Consulta grupos y listas de clase asignadas, registra, modifica y consolida calificaciones parciales/finales de sus estudiantes.
- **Coordinador Académico:** Configura planes de estudio, programas, asignaturas, periodos académicos, grupos, cupos, asigna docentes y aulas, supervisa matrículas y genera reportes de gestión.
- **Administrador del Sistema:** Gestiona el ciclo de vida de usuarios (altas, bajas, reseteos), asigna roles y permisos globales, consulta pistas de auditoría y monitorea la salud técnica de la plataforma.

Actores Secundarios (Sistemas o procesos automáticos de soporte)

- **Servicio de Auditoría / Logger:** Captura de forma asíncrona o transaccional los eventos críticos del sistema (autenticación, matrículas, cambios de notas).
- **Servicio de Copias de Seguridad (Backup Engine):** Ejecuta y valida las rutinas de respaldo programado de la base de datos y archivos.
- **Servicio de Notificaciones (Email/SMS):** Envía confirmaciones de matrícula, alertas de cambio de notas o enlaces de recuperación de credenciales.

P1-02

2. Supuestos Razonables de Arquitectura y Negocio

Dado que el alcance inicial no detalla toda la casuística operativa, se asumen las siguientes premisas de ingeniería:

1. **Ventanas de Tiempo Operativas:** Las matrículas y el registro de calificaciones están acotados por fechas límite definidas por periodo. Fuera de esos rangos, el sistema entra en modo solo lectura para estudiantes y docentes.
2. **Esquema de Calificaciones:** Se asume un modelo ponderado estándar (porcentajes de cortes o evaluaciones parciales que suman 100%) con un rango numérico predeterminado (ej. 0.0 a 5.0) y un umbral aprobatorio configurable.
3. **Conflictos de Horario:** El sistema no debe permitir que un estudiante o un docente esté en dos aulas/grupos en la misma franja horaria.
4. **Modelo de Concurrencia en Matrículas:** Aunque el RNF-02 pide 300 usuarios concurrentes generales, la matrícula genera picos agresivos de contención de datos (*race conditions* en los últimos cupos). El motor de persistencia requerirá control de concurrencia optimista/pesimista o colas de procesamiento.
5. **Mecanismo de Desactivación Lógica (*Soft Delete*):** La desactivación de estudiantes y docentes (RF-03, RF-04) no implica eliminación física de la base de datos, para no romper la integridad referencial histórica de matrículas y actas.
6. **Evolución a Autenticación Federada:** Aunque inicialmente se pide credenciales propias (usuario/contraseña), la arquitectura debe desacoplar el proveedor de identidad para soportar en el futuro SSO institucional (OAuth2 / SAML / Microsoft 365 / Google Workspace).

P1-03

3. Restricciones Técnicas y de Negocio

Tipo	Restricción	Impacto Arquitectónico
Técnica	Acceso exclusivo vía navegadores web modernos (RNF-12).	Se descarta el desarrollo de clientes nativos de escritorio; se exige arquitectura Web Responsive (SPA/SSR).
Técnica	Interfaces documentadas mediante API REST (RNF-08).	Desacoplamiento estricto entre Front-End y Back-End; adopción de estándares tipo OpenAPI/Swagger.
Técnica	Concurrencia de 300 usuarios con latencia $\leq$ 3 seg al percentil 95 (RNF-02).	Exige optimización de índices en BD, consultas paginadas, *caching* (ej. Redis) para catálogos y pooling de conexiones.
Técnica	Canales cifrados (HTTPS) y *hashing* de contraseñas (RNF-01).	Obliga a terminar TLS en balanceador o proxy inverso y usar algoritmos de derivación de claves robustos (Argon2id o BCrypt).
Negocio	Disponibilidad mensual mínima del 99.5% (RNF-03).	Máximo tolerado de ≈ 3.6 horas de caída no programada al mes; demanda balanceo de carga, monitoreo activo de *health checks* y despliegues sin interrupción (*zero-downtime*).
Negocio	Trazabilidad legal y regulatoria (RNF-10, RNF-11, RF-12).	Obligatoriedad de registros de auditoría inmutables (WORM) para alteraciones de calificaciones por implicaciones legales.

P1-04

4. Reglas de Negocio Implícitas Extraídas

A partir del análisis profundo de los RF y RNF, se extraen las siguientes reglas que condicionan la lógica del dominio:

- **RN-01 (Integridad de Cupos / Anti-Overbooking):** La asignación de cupos debe ser atómica. Ningún grupo puede tener más estudiantes matriculados que su límite de cupo configurado, incluso ante solicitudes simultáneas concurrentes (RNF-07).
- **RN-02 (Unicidad de Matrícula por Asignatura):** Un estudiante no puede estar matriculado activamente en más de un grupo de la misma asignatura dentro del mismo periodo académico.
- **RN-03 (Condición de Estado para Operar):** Solo los estudiantes con estado `Activo` pueden ser matriculados. Solo los docentes con estado `Activo` pueden ser asociados a la creación de grupos de un periodo.
- **RN-04 (Principio de Menor Privilegio en Calificaciones):** Un docente solo puede registrar y editar notas de estudiantes que pertenezcan a los grupos donde él figure formalmente como docente titular.
- **RN-05 (Cierre y Congelamiento de Calificaciones):** Una vez cerrado el periodo de notas por el Coordinador, el docente pierde el permiso de edición. Cualquier cambio posterior requiere una solicitud formal y autorización de nivel administrativo con registro de auditoría estricto (RF-12).
- **RN-06 (No Exposición de Datos Sensibles / Privacidad):** Un estudiante únicamente puede ver su propia información de notas y horarios; no puede acceder a las listas completas ni a los resultados de sus compañeros (RNF-11).
- **RN-07 (Inmutabilidad de Auditoría):** Las pistas de auditoría son de solo adición (*append-only*); ningún usuario, ni siquiera el Administrador, puede modificar o eliminar los logs de auditoría generados.

P1-05

5. Clasificación de RNF según Atributos de Calidad (ISO/IEC 25010)

Requerimiento	Atributo de Calidad Principal	Subcaracterística / Métrica	Enfoque de Diseño Arquitectónico
RNF-01 Seguridad	**Seguridad**	Confidencialidad e Integridad	Cifrado en tránsito (TLS 1.3), cifrado en reposo para credenciales (BCrypt/Argon2) y RBAC a nivel de API.
RNF-02 Rendimiento	**Eficiencia de Desempeño**	Comportamiento temporal y Capacidad	Paginación en servidor, indexación de BD, CDN para estáticos y capa de caché para lecturas repetitivas.
RNF-03 Disponibilidad	**Fiabilidad**	Disponibilidad operativa	Redundancia a nivel web/API, auto-recuperación de contenedores y *health checks* automatizados.
RNF-04 Escalabilidad	**Capacidad / Escalabilidad**	Escalabilidad horizontal	Estado de sesión desacoplado (Stateless APIs con JWT o sesiones centralizadas en Redis).
RNF-05 Mantenibilidad	**Mantenibilidad**	Modularidad, Reusabilidad y Analizabilidad	Arquitectura multicapa (Clean Architecture o Hexagonal), principios SOLID, inyección de dependencias.
RNF-06 Usabilidad	**Usabilidad**	Accesibilidad y Adaptabilidad	Diseño Mobile-First, cumplimiento de pautas WCAG 2.1 (AA), navegación semántica e interactividad accesible.
RNF-07 Integridad	**Fiabilidad / Integridad**	Consistencia transaccional	Transacciones ACID a nivel de base de datos relacional para procesos de matrícula y actas de notas.
RNF-08 Interoperabilidad	**Interoperabilidad**	Coexistencia e Intercambio de datos	Arquitectura orientada a APIs RESTful estándar, serialización JSON y documentación contractual con OpenAPI.
RNF-09 Respaldo	**Fiabilidad**	Recuperabilidad	Estrategia de respaldos programados (RPO/RTO definidos), réplicas y pruebas periódicas de restauración (*Disaster Recovery*).
RNF-10 Trazabilidad	**Mantenibilidad / Auditabilidad**	Analizabilidad	Mapeo bidireccional entre código, configuración y esquemas de datos; uso de *Correlation IDs* en trazas.
RNF-11 Privacidad	**Seguridad**	Confidencialidad	Control de acceso basado en atributos (ABAC/RBAC) a nivel de repositorio de datos y filtros de consulta por ID de usuario.
RNF-12 Compatibilidad	**Portabilidad**	Coexistencia y Adaptabilidad	Pruebas de compatibilidad multi-navegador (Chromium, Gecko, WebKit) y diseño responsivo basado en CSS Grid / Flexbox.

P2 · Estilo arquitectónico

Con base en el análisis anterior, propón el estilo arquitectónico para el SGA.
Compara al menos tres opciones: arquitectura en capas, monolito modular y microservicios.
Para cada una evalúa: complejidad de desarrollo y operación, rendimiento con 300 usuarios concurrentes, disponibilidad del 99,5 %, escalabilidad, mantenibilidad y consistencia en la matrícula ante solicitudes simultáneas.
Al final recomienda una opción y justifícala frente a los requerimientos no funcionales. Responde en español con una tabla comparativa.

P2-01

1. Evaluación de los Estilos Arquitectónicos

A. Arquitectura en Capas Tradicional (N-Tier: Presentación → Lógica de Negocio → Acceso a Datos)

- **Complejidad:** Muy baja en desarrollo y operación. Curva de aprendizaje mínima para el equipo.
- **Rendimiento (300 concurrentes):** Cumple con el objetivo sin problemas si la base de datos relacional está adecuadamente indexada.
- **Disponibilidad (99,5 %):** Alcanzable mediante un balanceador de carga que distribuya tráfico entre 2 o más instancias de la aplicación.
- **Escalabilidad:** Escalamiento horizontal del bloque completo. Ineficiente si solo el módulo de matrícula sufre congestión.
- **Mantenibilidad: Riesgo crítico.** Con el paso del tiempo, las capas monolíticas tradicionales tienden al fenómeno de *"Gran Bola de Lodo"* (*Big Ball of Mud*). La lógica de matrícula, notas y auditoría suele acoplarse fuertemente en las mismas entidades y servicios compartidos.
- **Consistencia en Matrícula:** Alta. Permite transacciones ACID locales directas en una única base de datos.

B. Monolito Modular (Modular Monolith)

- **Complejidad:** Media en desarrollo (exige disciplina de diseño para delimitar contextos y módulos independientes con APIs internas estrictas) y baja en operación (se empaqueta y despliega como un único artefacto o contenedor).
- **Rendimiento (300 concurrentes): Óptimo.** La comunicación intermodular ocurre en memoria (*in-process method calls*), eliminando la latencia de red y la sobrecarga de serialización HTTP/JSON. Supera con holgura la meta de < 3 s.
- **Disponibilidad (99,5 %):** Muy alta. Dos réplicas activas detrás de un balanceador (ej. Nginx o AWS ALB) toleran caídas de nodo sin interrumpir el servicio.
- **Escalabilidad:** Escalamiento horizontal eficiente del artefacto. Si a futuro un módulo específico (ej. Matrícula) requiere independizarse, sus límites limpios facilitan su extracción sin reescribir el sistema.
- **Mantenibilidad: Excelente.** Separa responsabilidades por dominio de negocio (Módulo Estudiantes, Módulo Calificaciones, Módulo Matrícula, Módulo Auditoría), garantizando alta cohesión y bajo acoplamiento sin la sobrecarga operativa de los microservicios.
- **Consistencia en Matrícula: Excelente.** Gestiona la contención de cupos y concurrencia mediante transacciones ACID nativas y aislamiento a nivel de base de datos relacional (bloqueo pesimista o selectivo por fila).

C. Microservicios (Servicios distribuidos independientes con BD por servicio)

- **Complejidad: Extremadamente alta.** Requiere infraestructura avanzada (orquestador tipo Kubernetes, *service mesh*, *distributed tracing*, pasarela de API) y gestión compleja de despliegues independientes. Sobrediseño (*over-engineering*) para el tamaño de la institución.
- **Rendimiento (300 concurrentes):** Puede verse penalizado por saltos de red (*network hops*) y latencia de serialización en consultas compuestas (ej. generar el consolidado de matrícula y notas de un estudiante).
- **Disponibilidad (99,5 %):** Teóricamente alta por aislamiento de fallas, pero en la práctica vulnerable a fallas en cascada y dependencia de la red si no se implementan patrones complejos de resiliencia (*circuit breaker*, reintentos).
- **Escalabilidad:** Máxima a nivel granular, permitiendo asignar recursos únicamente al servicio de matrículas.
- **Mantenibilidad:** Alta a nivel de código de cada servicio, pero muy difícil de trazar y mantener en cambios que cruzan contratos de múltiples servicios.
- **Consistencia en Matrícula: Crítica y de alto riesgo.** Al requerir bases de datos separadas, la validación de cupos y cruces de horario obligaría a implementar consistencia eventual y patrones Saga. Esto incrementa dramáticamente el riesgo de *race conditions* y sobrecupo (*overbooking*) en periodos pico de matrícula.

P2-02

2. Tabla Comparativa

Criterio de Evaluación	Arquitectura en Capas	Monolito Modular	Microservicios
Complejidad de Desarrollo	**Baja** (un solo proyecto, patrones simples).	**Media** (diseño DDD, contratos internos).	**Muy Alta** (manejo de sistemas distribuidos).
Complejidad Operativa (DevOps)	**Baja** (un único artefacto a desplegar).	**Baja** (un único artefacto o contenedor).	**Extremadamente Alta** (K8s, redes, observabilidad distribuida).
Rendimiento (300 conc. / $\leq$ 3 s)	**Bueno** (suficiente para la carga).	**Excelente** (llamadas en memoria, sin latencia de red).	**Moderado/Variable** (latencia de red inter-servicio).
Disponibilidad (99,5 %)	**Media-Alta** (2 nodos + Load Balancer).	**Alta** (2-3 réplicas + Load Balancer, rápido arranque).	**Alta** (aislamiento de fallos, pero depende de la red).
Escalabilidad (RNF-04)	**Media** (escala todo el bloque).	**Buena** (escala todo el bloque; desacoplado internamente).	**Excelente** (escala servicios específicos bajo demanda).
Mantenibilidad (RNF-05)	**Baja** (riesgo de acoplamiento espagueti).	**Alta** (módulos cohesivos y aislados por dominio).	**Media-Alta** (servicios pequeños, pero contratos complejos).
Consistencia en Matrícula (RNF-07)	**Alta** (transacciones ACID directas).	**Óptima** (ACID relacional, sin complejidad distribuida).	**Deficiente/Compleja** (requiere Sagas y consistencia eventual).

P2-03

3. Recomendación Arquitectónica y Justificación

Estilo Recomendado: Monolito Modular (Modular Monolith)

Se recomienda adoptar una arquitectura de **Monolito Modular con persistencia relacional única y partición lógica de esquemas**, complementada con una interfaz web desacoplada (SPA o SSR) consumiendo una API REST.

Justificación frente a los Requerimientos No Funcionales:

1. **Integridad y Consistencia (RNF-07):** El proceso de matrícula con validación de cupos concurrentes y prevención de duplicados es el mayor riesgo del SGA. El monolito modular permite ejecutar transacciones atómicas (**ACID**) en la base de datos relacional (ej. PostgreSQL con `SELECT ... FOR UPDATE` o control de concurrencia optimista), garantizando que jamás se exceda un cupo ni se matricule una asignatura duplicada, eliminando la fragilidad de la consistencia eventual de los microservicios.
2. **Rendimiento y Concurrencia (RNF-02):** Un pico de 300 usuarios concurrentes es una carga ligera para la capacidad de cómputo moderna. Al mantener los módulos (Estudiantes, Asignaturas, Matrícula, Calificaciones) dentro del mismo proceso de ejecución, las llamadas entre módulos son instantáneas en memoria, garantizando holgadamente respuestas en menos de 3 segundos para el 95 % de las peticiones.
3. **Mantenibilidad y Modularidad (RNF-05):** A diferencia de la arquitectura en capas genérica, el monolito modular define barreras estrictas de dominio (Contextos Delimitados de DDD). El código de calificaciones no puede tocar directamente las tablas o entidades de matrícula, sino solo a través de interfaces públicas bien definidas, facilitando refactorizaciones, pruebas unitarias y reduciendo la deuda técnica.
4. **Escalabilidad y Crecimiento Futuro (RNF-04):** Permite atender el crecimiento institucional añadiendo réplicas idénticas detrás de un balanceador de carga. Además, si dentro de varios años el módulo de matrícula o el de autenticación requiere escalar independientemente, el aislamiento de sus interfaces permite extraerlo hacia un microservicio autónomo con un costo de refactorización mínimo.
5. **Disponibilidad y Recuperabilidad (RNF-03, RNF-09):** Desplegar 2 o 3 réplicas del monolito en contenedores (ej. Docker) detrás de un proxy/balanceador garantiza el 99,5 % de disponibilidad. La gestión de copias de seguridad y procedimientos de restauración (RTO/RPO) sobre una única base de datos relacional es directa y confiable, sin lidiar con restauraciones desincronizadas entre múltiples almacenes de datos.
6. **Seguridad y Auditoría Centralizada (RNF-01, RNF-10, RNF-11):** La aplicación de políticas de roles y permisos (RBAC) y el registro transaccional de auditoría (RF-12) se implementan mediante interceptores (*middleware*) centralizados y atómicos en el ciclo de vida de la petición, garantizando que ninguna mutación de notas quede sin registro auditable.

P2-04

P3 · Componentes y vista lógica

Para la arquitectura que recomendaste, describe la vista lógica:
los módulos o componentes principales, la responsabilidad de cada uno, las interfaces que exponen (por ejemplo, endpoints REST principales) y las dependencias entre ellos.
Debe quedar claro cómo se organizan autenticación y autorización, gestión académica, matrículas, calificaciones, reportes y auditoría.
Genera además el diagrama en sintaxis Mermaid.

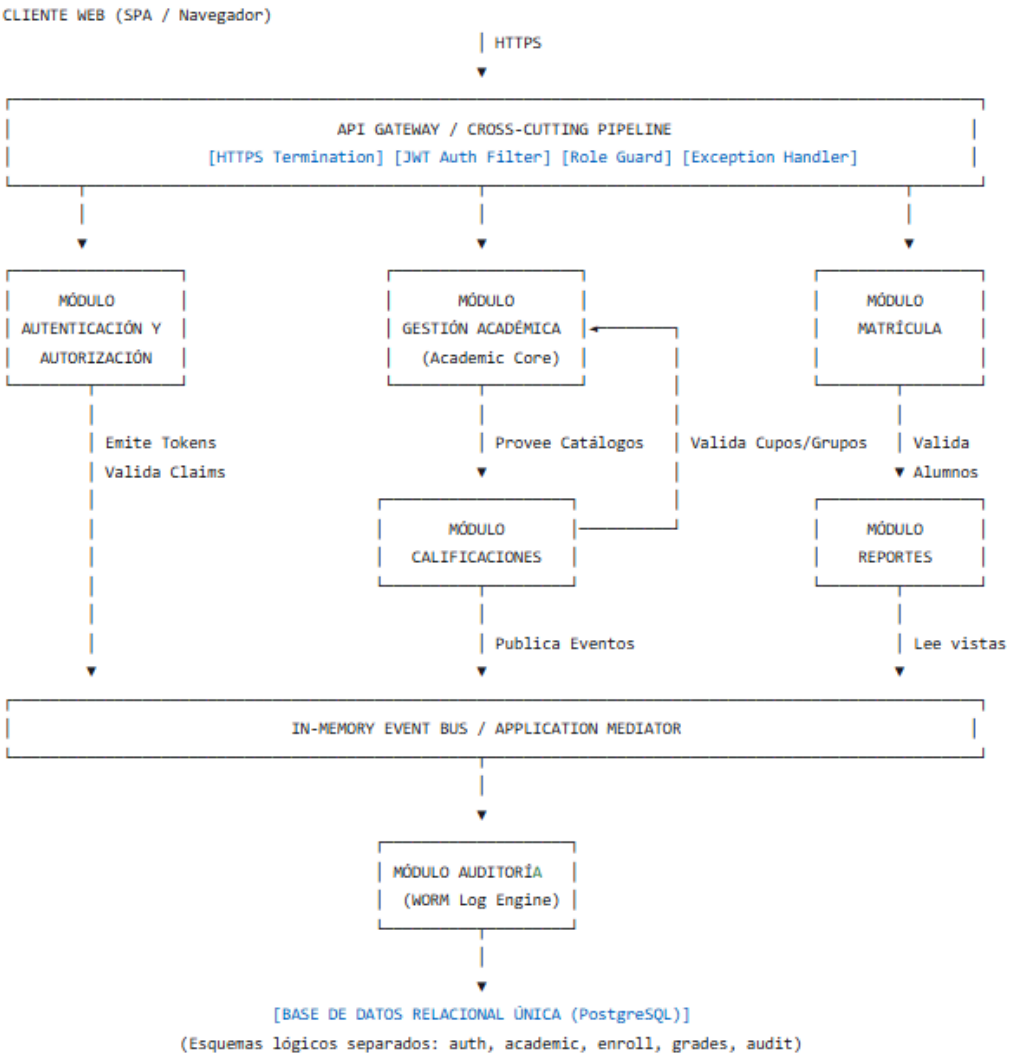
P3-01

El diseño se estructura bajo principios de *Domain-Driven Design* (DDD) mediante **Contextos Delimitados (*Bounded Contexts*)**, donde cada módulo es un paquete autónomo en el mismo proceso de despliegue, comunicándose hacia el exterior mediante APIs REST y entre sí mediante contratos/interfaces en memoria o eventos de dominio.

---

1. Componentes y Módulos Principales

Code



P3-02

2. Detalle de Módulos: Responsabilidades, Interfaces y Dependencias

2.1. Módulo de Autenticación y Autorización (IAM)

- **Responsabilidad:** Gestión del ciclo de vida de credenciales (hashing Argon2id), emisión y verificación de JSON Web Tokens (JWT), resolución de roles (`ADMINISTRADOR`, `COORDINADOR`, `DOCENTE`, `ESTUDIANTE`) y permisos.
- **Endpoints REST:**
  - `POST /api/v1/auth/login` (Retorna JWT firmado + *refresh token*).
  - `POST /api/v1/auth/logout` (Invalida el token en lista negra / Redis).
  - `PUT /api/v1/auth/users/{id}/roles` (Solo `ADMINISTRADOR`: asignación de perfiles).
- **Interfaz Interna (In-Memory Service):**
  - `ISecurityContext`: Expone método `GetCurrentUser(): UserContext` con `userId`, `roles`, y `tenant/metadata`.
- **Dependencias:** Ninguna (módulo base).

2.2. Módulo de Gestión Académica (Academic Core)

- **Responsabilidad:** Administración de entidades fundamentales: Estudiantes, Docentes, Programas, Asignaturas, Periodos, Aulas y Grupos con sus horarios y límites de cupo (RF-03, RF-04, RF-05, RF-06).
- **Endpoints REST:**
  - `POST|GET|PUT /api/v1/academic/programs` y `/courses` (Gestión de oferta).
  - `POST|GET|PUT /api/v1/academic/terms` (Configuración de periodos académicos).
  - `POST|GET|PUT /api/v1/academic/groups` (Creación de grupos, asignación docente y definición de cupos).
  - `POST|PUT /api/v1/academic/students` y `/professors` (Altas, bajas lógicas y modificaciones).
- **Interfaz Interna (In-Memory Service):**
  - `IAcademicQueryService`: Permite a otros módulos verificar si un estudiante está `Activo`, si un docente está asignado a un grupo específico o consultar los prerrequisitos de una asignatura.
- **Dependencias:** `ISecurityContext` (para validar roles administrativos/coordinadores).

2.3. Módulo de Matrícula (Enrollment Engine)

- **Responsabilidad:** Procesar la inscripción de asignaturas asegurando **consistencia transaccional estricta (ACID)**. Valida cupos en tiempo real, verifica que no existan cruces de horario, impide duplicidad de asignaturas y actualiza la ocupación del grupo (RF-07, RNF-07).
- **Endpoints REST:**
  - `POST /api/v1/enrollments` (Ejecuta la matrícula de uno o varios grupos).
  - `DELETE /api/v1/enrollments/{enrollmentId}/groups/{groupId}` (Cancelación de grupo).
  - `GET /api/v1/enrollments/my-schedule` (Horario del estudiante autenticado).
- **Interfaz Interna:**
  - `IEnrollmentQueryService`: Expone `GetEnrolledStudentsByGroup(groupId): List<StudentRef>` (usado por el Módulo de Calificaciones).
- **Dependencias:**
  - `IAcademicQueryService` (para validar estado del estudiante, validez del grupo y cupos máximos).
  - `IEventPublisher` (publica `StudentEnrolledEvent` hacia Auditoría).

2.4. Módulo de Calificaciones (Grades Management)

- **Responsabilidad:** Registro, edición y cálculo ponderado de notas parciales y finales. Bloquea modificaciones si el docente no es titular o si la ventana temporal del periodo cerró (RF-08, RF-09).
- **Endpoints REST:**
  - `GET /api/v1/grades/groups/{groupId}` (Lista de notas; acceso para el docente titular o coordinador).
  - `PUT /api/v1/grades/groups/{groupId}/evaluations` (Carga/actualización de notas por corte).
  - `GET /api/v1/grades/my-grades` (Solo lectura para el estudiante sobre sus notas publicadas).
- **Dependencias:**
  - `IAcademicQueryService` (valida asignación del docente al grupo y estado del periodo).
  - `IEnrollmentQueryService` (obtiene la lista oficial de matriculados del grupo).
  - `IEventPublisher` (publica `GradeModifiedEvent` con nota previa, nota nueva, autor y justificación hacia Auditoría).

P3-04

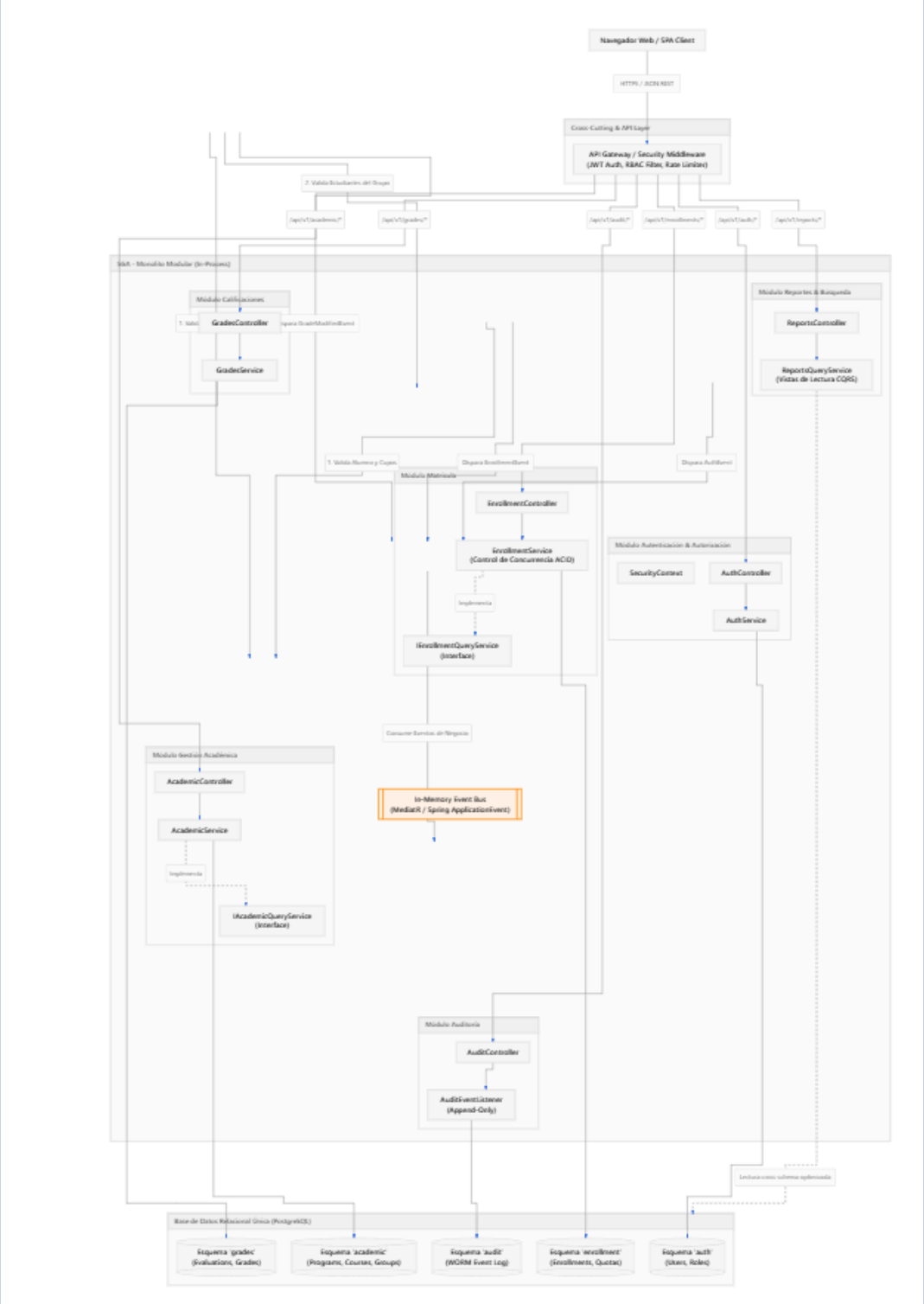
2.5. Módulo de Reportes y Búsqueda (Reporting & Search)

- **Responsabilidad:** Generación optimizada de consultas consolidadas, listas de clase, sábanas de notas y reportes institucionales en PDF/Excel sin degradar el procesamiento transaccional (RF-10, RF-11).
- **Endpoints REST:**
  - `GET /api/v1/reports/enrollments?termId=X&programId=Y`
  - `GET /api/v1/reports/grades-summary?termId=X&groupId=Y`
  - `GET /api/v1/search?q={term}&entity={student|teacher|group}`
- **Dependencias:** Consalta directamente las vistas de lectura en la base de datos relacional (patrón CQRS a nivel de base de datos usando vistas SQL optimizadas/réplicas de lectura).

2.6. Módulo de Auditoría (Audit Engine)

- **Responsabilidad:** Registro inmutable (*Append-Only*) de eventos sensibles: inicios de sesión, cambios de roles, matrículas efectuadas y alteraciones de calificaciones (RF-12, RNF-10).
- **Endpoints REST:**
  - `GET /api/v1/audit/logs?entity=X&userId=Y&fromDate=Z` (Restringido exclusivamente al `ADMINISTRADOR`).
- **Mecanismo de Ingesta:**
  - Suscriptor desacoplado del `In-Memory Event Bus`. Escucha eventos asíncronos (`GradeModifiedEvent`, `UserLoggedInEvent`, `RoleChangedEvent`, `EnrollmentCompletedEvent`) y persiste de forma asíncrona pero confiable en la tabla `audit.logs`.
- **Dependencias:** Ninguna (módulo sumidero de eventos).

P3-05


Navegador Web / SPA Client
HTTPS / JSON REST
Cross-Cutting & API Layer
API Gateway / Security Middleware
(JWT Auth, RBAC Filter, Rate Limiter)
SGA - Monolito Modular (In-Process)
Módulo Calificaciones
GradesController
GradesService
Módulo Reportes & Búsqueda
ReportsController
ReportsQueryService
(Vistas de Lectura CQRS)
Módulo Matrícula
EnrollmentController
EnrollmentService
(Control de Concurrencia ACID)
IEnrollmentQueryService
(Interface)
Módulo Autenticación & Autorización
SecurityContext
AuthController
AuthService
Módulo Gestión Académica
AcademicController
AcademicService
IAcademicQueryService
(Interface)
In-Memory Event Bus
(MediatR / Spring ApplicationEvent)
Módulo Auditoría
AuditController
AuditEventListener
(Append-Only)
Base de Datos Relacional Única (PostgreSQL)
Esquema 'grades'
(Evaluations, Grades)
Esquema 'academic'
(Programs, Courses, Groups)
Esquema 'audit'
(WORM Event Log)
Esquema 'enrollment'
(Enrollments, Quotas)
Esquema 'auth'
(Users, Roles)

P3-06

4. Patrones de Colaboración Clave

1. **Matrícula Concurrente Atómica (RNF-07):**
   Cuando `EnrollmentController` recibe una solicitud, abre una transacción ACID en el esquema `enrollment`. Mediante `IAcademicQueryService` confirma el estado del estudiante y consulta el cupo. Ejecuta un bloqueo pesimista (`SELECT ... FOR UPDATE`) sobre la fila del grupo para restar el cupo en el mismo commit. Si la transacción finaliza con éxito, emite un evento `StudentEnrolledEvent` al bus en memoria.
2. **Modificación de Notas con Trazabilidad Estricta (RF-08, RF-12):**
   `GradesService` recibe el cambio de nota. Valida con `IAcademicQueryService` que el usuario autenticado sea el docente titular asignado. Al actualizar la nota en el esquema `grades`, se genera sincrónicamente un evento con el payload `{estudianteId, grupoId, notaAnterior, notaNueva, docenteId, justificacion}`. El `AuditEventListener` procesa el evento y escribe en el esquema inmutable `audit`.
3. **Aislamiento de Módulos:**
   Ningún módulo ejecuta consultas SQL directas contra las tablas de otro módulo (salvo el módulo de *Reportes*, que tiene permisos de solo lectura optimizados para analítica). Cualquier validación cruzada entre *Matrículas* y *Calificaciones* se realiza estrictamente a través de las interfaces (`IAcademicQueryService`, `IEnrollmentQueryService`).

P4 · Modelo de datos conceptual

Propón el modelo de datos conceptual del SGA con las entidades: Usuario, Rol, Estudiante, Docente, Programa, Asignatura, PeriodoAcadémico, Grupo, Matrícula y Calificación, más las que consideres necesarias (por ejemplo, para horarios o auditoría).
Indica los atributos principales y las cardinalidades de cada relación, y explica las restricciones que evitan matrículas duplicadas.
Genera el diagrama entidad-relación en sintaxis Mermaid (erDiagram)

P4-01

1. Entidades Principales y Atributos

1. **Usuario:** Credenciales y estado global de acceso.
   - `id_usuario` (PK), `email` (UK), `password_hash`, `nombre`, `apellido`, `estado` (Activo/Inactivo), `fecha_creacion`.
2. **Rol:** Catálogo de permisos.
   - `id_rol` (PK), `codigo_rol` (UK: ADMIN, COORDINADOR, DOCENTE, ESTUDIANTE), `descripcion`.
3. **UsuarioRol:** Tabla asociativa para soportar asignación múltiple de roles (M:N).
   - `id_usuario` (PK, FK), `id_rol` (PK, FK).
4. **Estudiante:** Extensión de perfil académico del usuario.
   - `id_estudiante` (PK), `id_usuario` (FK, UK, 1:1), `codigo_estudiante` (UK), `id_programa` (FK), `estado_academico` (Regular, Egresado, Suspendido).
5. **Docente:** Extensión del perfil docente del usuario.
   - `id_docente` (PK), `id_usuario` (FK, UK, 1:1), `codigo_docente` (UK), `especialidad`, `tipo_contrato`.
6. **Programa:** Oferta académica institucional.
   - `id_programa` (PK), `codigo_programa` (UK), `nombre`, `nivel_academico` (Pregrado, Posgrado), `creditos_totales`.
7. **Asignatura:** Unidades curriculares asociadas a un plan de estudios.
   - `id_asignatura` (PK), `codigo_asignatura` (UK), `id_programa` (FK), `nombre`, `creditos`, `semestre_nivel`.
8. **PeriodoAcademico:** Ciclos lectivos regulares o vacacionales.
   - `id_periodo` (PK), `codigo_periodo` (UK: ej. "2025-1"), `fecha_inicio`, `fecha_fin`, `estado` (Planificacion, Abierto, Cerrado).
9. **Grupo:** Instancia concreta de una asignatura dictada en un periodo.
   - `id_grupo` (PK), `id_asignatura` (FK), `id_periodo` (FK), `id_docente` (FK), `codigo_grupo` (ej. "G01"), `cupo_maximo`, `cupo_ocupado`.
10. **Horario:** Franjas de clase asignadas a un grupo.
    - `id_horario` (PK), `id_grupo` (FK), `dia_semana` (1-7), `hora_inicio`, `hora_fin`, `aula_o_enlace`.
11. **Matricula (Cabecera):** Formalización de la inscripción del estudiante en un periodo académico.
    - `id_matricula` (PK), `id_estudiante` (FK), `id_periodo` (FK), `fecha_registro`, `estado` (Inscrita, Retirada).
12. **DetalleMatricula (Línea de Matrícula):** Asignación específica a un grupo del periodo.
    - `id_detalle` (PK), `id_matricula` (FK), `id_grupo` (FK), `id_asignatura` (FK denormalizada para integridad), `estado` (Matriculado, Retirado).
13. **Calificacion:** Evaluaciones y notas por corte.
    - `id_calificacion` (PK), `id_detalle` (FK), `nombre_corte` (ej. "Parcial 1"), `porcentaje` (ej. 30%), `nota_obtenida`, `fecha_registro`, `id_docente_calificador` (FK).
14. **Auditoria:** Pistas inmutables de operaciones sensibles (WORM).
    - `id_auditoria` (PK, BigInt), `id_usuario` (FK), `accion` (LOGIN, REGISTRO_NOTA, CAMBIO_ROL, MATRICULA), `entidad_afectada`, `id_registro_afectado`, `valores_anteriores` (JSONB), `valores_nuevos` (JSONB), `fecha_hora`, `ip_origen`.

P4-02

2. Cardinalidades de las Relaciones

- **Usuario – Rol:** `M:N` (Un usuario puede tener uno o más roles; un rol aplica a múltiples usuarios) vía `UsuarioRol`.
- **Usuario – Estudiante / Docente:** `1:0..1` (Herencia por composición: un usuario puede o no ser estudiante o docente).
- **Programa – Estudiante:** `1:N` (Un programa tiene muchos estudiantes; un estudiante pertenece a un programa principal).
- **Programa – Asignatura:** `1:N` (Un programa estructura muchas asignaturas).
- **Asignatura – Grupo:** `1:N` (Una asignatura se oferta en uno o varios grupos por periodo).
- **PeriodoAcademico – Grupo:** `1:N` (Un periodo agrupa múltiples cursos abiertos).
- **Docente – Grupo:** `1:N` (Un docente dicta cero o varios grupos; un grupo tiene un docente responsable).
- **Grupo – Horario:** `1:N` (Un grupo tiene varias sesiones semanales de clase).
- **Estudiante – Matricula:** `1:N` (Un estudiante se matricula a lo largo de varios periodos).
- **PeriodoAcademico – Matricula:** `1:N` (Un periodo contiene muchas matrículas).
- **Matricula – DetalleMatricula:** `1:N` (Una matrícula inscribe varias asignaturas/grupos).
- **Grupo – DetalleMatricula:** `1:N` (Un grupo tiene muchos estudiantes inscritos).
- **DetalleMatricula – Calificacion:** `1:N` (Una asignatura matriculada acumula varios cortes de nota).
- **Usuario – Auditoria:** `1:N` (Un usuario genera múltiples eventos de trazabilidad).

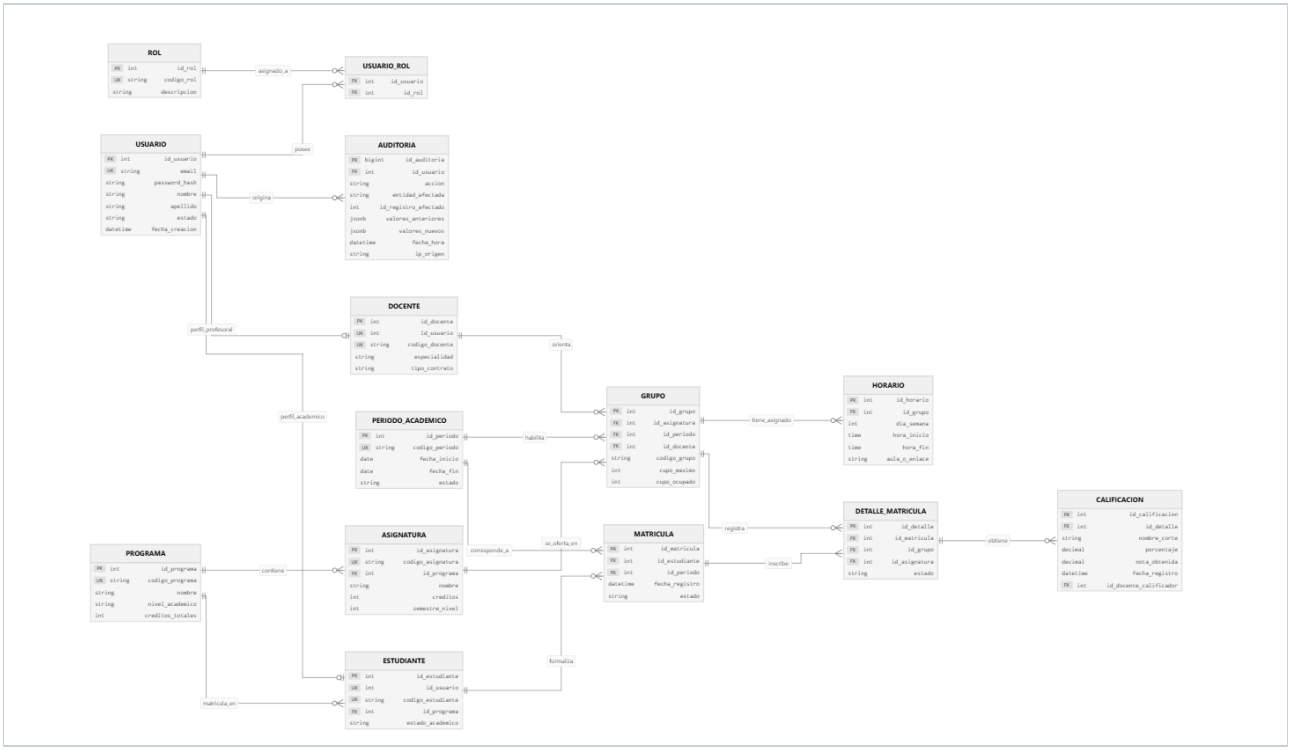
P4-03

3. Restricciones Técnicas para Evitar Matrículas Duplicadas y Sobrecupo

Para asegurar el **RNF-07 (Integridad y consistencia)**, el sistema no se confía únicamente de la validación en código Back-End; impone **restricciones formales a nivel de Base de Datos**:

1. **Unicidad de Matrícula por Periodo (Nivel Cabecera):**
   - Restricción: `UNIQUE (id_estudiante, id_periodo)` en la tabla `Matricula`.
   - *Efecto:* Impide que un estudiante abra más de un proceso de matrícula para el mismo periodo académico.
2. **Prevención de Asignatura Duplicada en Diferentes Grupos (Nivel Detalle):**
   - Restricción: `UNIQUE (id_matricula, id_asignatura)` en la tabla `DetalleMatricula`.
   - *Efecto:* Aunque un estudiante intente registrar el Grupo 1 y el Grupo 2 de la misma asignatura (ej. "Cálculo I"), la base de datos abortará la transacción por colisión de clave única.
3. **Prevención de Inscripción Duplicada en el Mismo Grupo:**
   - Restricción: `UNIQUE (id_matricula, id_grupo)` en `DetalleMatricula`.
   - *Efecto:* Evita inserciones redundantes concurrentes en el mismo grupo.
4. **Control Estricto de Cupo Máximo (Anti-Sobrecupo / Overbooking):**
   - Restricción de verificación en `Grupo`: `CHECK (cupo_ocupado <= cupo_maximo)`.
   - *Mecanismo Transaccional:* En el momento de la inscripción, el Back-End ejecuta un bloqueo de fila (`SELECT cupo_ocupado, cupo_maximo FROM Grupo WHERE id_grupo = :id FOR UPDATE`), valida disponibilidad, incrementa `cupo_ocupado = cupo_ocupado + 1` e inserta el `DetalleMatricula` bajo la misma transacción ACID atómica.
5. **Prevención de Cruce de Horarios (Solapamiento Temporal):**
   - Validación a nivel de servicio y soportada opcionalmente mediante restricciones de exclusión temporal en PostgreSQL (`EXCLUDE USING GIST` sobre rangos de tiempo y días), validando que no coincidan dos grupos con `(dia_semana, hora_inicio, hora_fin)` para el mismo estudiante.

P4-04


ROL
id_rol
codigo_rol
descripcion
asignado_a
USUARIO_ROL
id_usuario
id_rol
USUARIO
id_usuario
email
password_hash
nombre
apellido
estado
fecha_creacion
posee
origina
AUDITORIA
id_auditoria
id_usuario
accion
entidad_afectada
id_registro_afectado
valores_anteriores
valores_nuevos
fecha_hora
ip_origen
DOCENTE
id_docente
id_usuario
codigo_docente
especialidad
tipo_contrato
perfil_profesoral
orienta
perfil_academico
PERIODO_ACADEMICO
id_periodo
codigo_periodo
fecha_inicio
fecha_fin
estado
habilita
GRUPO
id_grupo
id_asignatura
id_periodo
id_docente
codigo_grupo
cupo_maximo
cupo_ocupado
tiene_asignado
HORARIO
id_horario
id_grupo
dia_semana
hora_inicio
hora_fin
aula_o_enlace
PROGRAMA
id_programa
codigo_programa
nombre
nivel_academico
creditos_totales
confiere
ASIGNATURA
id_asignatura
codigo_asignatura
id_programa
nombre
creditos
semestre_nivel
corresponde_a
se_oferta_en
MATRICULA
id_matricula
id_estudiante
id_periodo
fecha_registro
estado
registra
inscribe
DETALLE_MATRICULA
id_detalle
id_matricula
id_grupo
id_asignatura
estado
obtiene
CALIFICACION
id_calificacion
id_detalle
nombre_corte
porcentaje
nota_obtenida
fecha_registro
id_docente_calificador
ESTUDIANTE
id_estudiante
id_usuario
codigo_estudiante
id_programa
estado_academico
matricula_en
formaliza

P4-05

P5 · Secuencia de la matrícula

Modela el diagrama de secuencia del proceso de matrícula de un estudiante en un grupo.
Incluye los participantes (estudiante o coordinador, interfaz web, API, módulos internos, base de datos y auditoría), el orden de los mensajes, la validación de cupo, la validación de matrícula duplicada y qué pasa si dos solicitudes llegan al mismo tiempo por el último cupo.
Genera el diagrama en sintaxis Mermaid (sequenceDiagram)

P5-01

A continuación, se detalla el diseño de interacción y concurrencia para el proceso de matrícula de una asignatura/grupo.

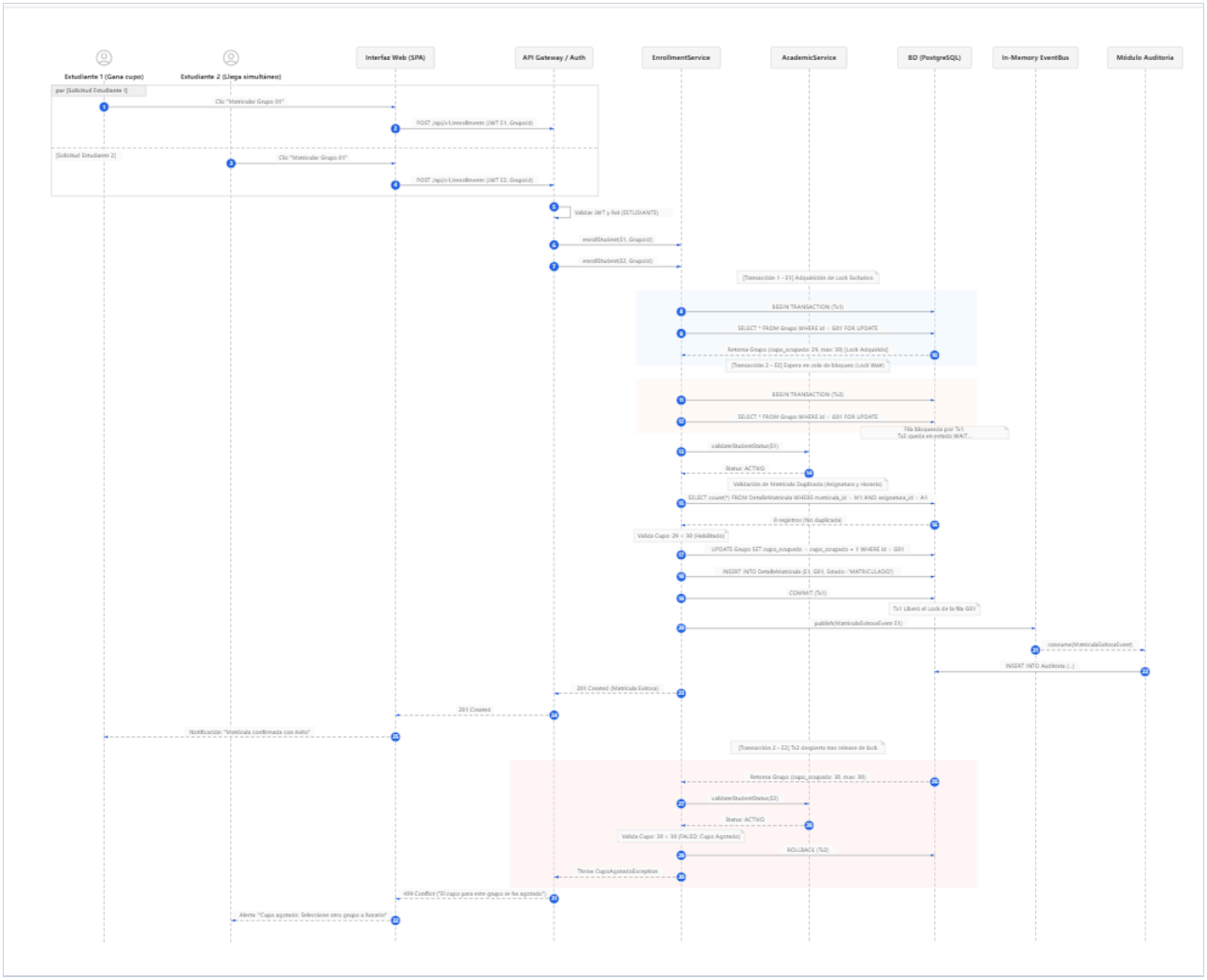
Para resolver de forma estricta el **RNF-07 (Integridad y consistencia)** y evitar sobrecupos (*overbooking*), la arquitectura implementa un mecanismo de **bloqueo pesimista a nivel de fila (`SELECT ... FOR UPDATE`)** dentro de una transacción ACID en PostgreSQL.

---

1. Descripción del Flujo y Resolución de Concurrencia

1. **Autenticación y Autorización:** La API intercepta las solicitudes, valida la firma del token JWT y comprueba que los usuarios posean el rol `ESTUDIANTE` (o `COORDINADOR`) habilitado.
2. **Inicio de Transacción Atómica:** Se abre una transacción en la base de datos para cada solicitud.
3. **Escenario de Concurrencia Crítica (Último Cupo):**
   - Las solicitudes de **Estudiante 1** y **Estudiante 2** llegan prácticamente al mismo milisegundo por el cupo 30 de 30 (`cupo_ocupado = 29`, `cupo_maximo = 30`).
   - La **Transacción 1 (Tx1)** adquiere primero el cerrojo exclusivo sobre el registro del grupo mediante `SELECT ... FOR UPDATE`.
   - La **Transacción 2 (Tx2)** entra en **estado de espera bloqueada (*WAIT*)** en el motor de base de datos hasta que Tx1 libere el cerrojo.
4. **Validaciones en Tx1:**
   - **Duplicidad:** Verifica que el Estudiante 1 no tenga ya matriculada esa asignatura en el periodo actual ni cruce de horario.
   - **Cupo:** Evalúa `cupo_ocupado (29) < cupo_maximo (30)`. Es válido.
   - **Persistencia:** Incrementa `cupo_ocupado` a 30, inserta el registro en `DetalleMatricula` y ejecuta `COMMIT`.
   - **Auditoría Asíncrona:** Emite el evento `MatriculaExitosaEvent` hacia el bus de eventos en memoria sin frenar la respuesta al usuario.
5. **Resolución de Tx2 (Solicitud Rechazada):**
   - Tras el `COMMIT` de Tx1, se libera el cerrojo y **Tx2 despierta**, leyendo inmediatamente el estado actualizado: `cupo_ocupado = 30`.
   - Tx2 evalúa la condición de cupo: `30 < 30` resulta **falso**.
   - Tx2 ejecuta `ROLLBACK` inmediato y el servicio arroja la excepción de dominio `CupoAgotadoException`.
   - La API responde al Estudiante 2 un código HTTP `409 Conflict`, garantizando **cero sobrecupo**.

P5-02


Estudiante 1 (Gana cupo)
Estudiante 2 (Llega simultáneo)
Interfaz Web (SPA)
API Gateway / Auth
EnrollmentService
AcademicService
BD (PostgreSQL)
In-Memory EventBus
Módulo Auditoría
par [Solicitud Estudiante 1]
Clic "Matricular Grupo 01"
POST /api/v1/enrollments (JWT E1, GrupoId)
[Solicitud Estudiante 2]
Clic "Matricular Grupo 01"
POST /api/v1/enrollments (JWT E2, GrupoId)
Validar JWT y Rol (ESTUDIANTE)
enrollStudent(E1, GrupoId)
enrollStudent(E2, GrupoId)
[Transacción 1 – E1] Adquisición de Lock Exclusivo
BEGIN TRANSACTION (Tx1)
SELECT * FROM Grupo WHERE id = G01 FOR UPDATE
Retorna Grupo (cupo_ocupado: 29, max: 30) [Lock Adquirido]
[Transacción 2 – E2] Espera en cola de bloqueo (Lock Wait)
BEGIN TRANSACTION (Tx2)
SELECT * FROM Grupo WHERE id = G01 FOR UPDATE
Fila bloqueada por Tx1
Tx2 queda en estado WAIT...
validateStudentStatus(E1)
Status: ACTIVO
Validación de Matrícula Duplicada (Asignatura y Horario)
SELECT count(*) FROM DetalleMatricula WHERE matricula_id = M1 AND asignatura_id = A1
0 registros (No duplicada)
Valida Cupo: 29 < 30 (Habilitado)
UPDATE Grupo SET cupo_ocupado = cupo_ocupado + 1 WHERE id = G01
INSERT INTO DetalleMatricula (E1, G01, Estado: 'MATRICULADO')
COMMIT (Tx1)
Tx1 Libera el Lock de la fila G01
publish(MatriculaExitosaEvent E1)
consume(MatriculaExitosaEvent)
INSERT INTO Auditoría (...)
201 Created (Matrícula Exitosa)
201 Created
Notificación: "Matrícula confirmada con éxito"
[Transacción 2 – E2] Tx2 despierta tras release de lock
Retorna Grupo (cupo_ocupado: 30, max: 30)
validateStudentStatus(E2)
Status: ACTIVO
Valida Cupo: 30 < 30 (FALSO: Cupo Agotado)
ROLLBACK (Tx2)
Throw CupoAgotadoException
409 Conflict ("El cupo para este grupo se ha agotado")
Alerta: "Cupo agotado. Seleccione otro grupo u horario"

P5-03

3. Consideraciones de Resiliencia y Rendimiento en el Flujo

1. **Tiempo de Retención de Bloqueos (*Lock Contention*):**
   - La transacción solo mantiene el bloqueo de fila durante las micro-operaciones de lectura y escritura (un tiempo típico de ≤ 15 ms). Esto garantiza que, incluso con múltiples estudiantes compitiendo simultáneamente, la cola de espera de la base de datos se desahogue rápidamente sin provocar agotamiento de conexiones en el pool de la aplicación.
2. **Prevención de *Deadlocks* (Bloqueos Mutuos):**
   - Cuando un estudiante inscribe varias asignaturas en un mismo envío (*batch enrollment*), el backend ordena previamente los `id_grupo` de forma ascendente antes de solicitar los bloqueos `FOR UPDATE`. Esto previene de forma determinista cualquier situación de abrazo mortal entre transacciones concurrentes.
3. **Desacoplamiento de la Auditoría:**
   - La escritura en la tabla `Auditoria` ocurre de manera asíncrona mediante el `EventBus`. Así, la persistencia del log de auditoría no retrasa la confirmación de la matrícula al estudiante ni retiene recursos de la transacción crítica de cupos.

User • 1:00 p.m.

Propón el modelo de despliegue del SGA: nodos o servicios donde se ejecutan la interfaz web, la API, la base de datos y los servicios complementarios.
Indica los canales de comunicación, los controles de seguridad (HTTPS, firewall, manejo de contraseñas, tokens), cómo se logra la disponibilidad del 99,5 % y cómo serían las copias de seguridad y la restauración.
Considera que es una institución con presupuesto moderado. Genera el diagrama en sintaxis Mermaid.

P5-04

P6 · Despliegue, seguridad y respaldo

Para una institución de educación superior con **presupuesto moderado**, la arquitectura de infraestructura debe evitar la complejidad y el alto costo de plataformas como Kubernetes multi-región, priorizando una estrategia **pragmática, altamente automatizada y de costo eficiente** basada en servicios administrados estándar o máquinas virtuales optimizadas con contenedores Docker.

1. Nodos y Servicios del Despliegue

Capa / Servicio	Componente Tecnológico	Propósito y Dimensionamiento
Edge / CDN	Cloudflare (Plan Free/Pro)	Terminación TLS, mitigación de DDoS, WAF básico y distribución de contenido estático (SPA). Costo casi nulo.
Frontend Web	Almacenamiento de Objetos (S3 / MinIO) + CDN	Aloja la interfaz web compilada (HTML5/JS/CSS). Elimina el consumo de CPU en servidores para servir archivos estáticos.
Balanceador de Carga	Nginx / HAProxy o Cloud Load Balancer	Proxy inverso que distribuye tráfico HTTPS hacia los nodos del backend y monitorea *health checks*.
Backend API (Monolito)	2 Instancias de Cómputo (VM o Contenedores)	Ejecutan el artefacto monolítico modular (Node.js/NestJS, Spring Boot o .NET 8). 2 vCPU y 4 GB RAM por nodo bastan para 300 concurrentes.
Caché y Mensajería	Redis (Instancia Standalone / Docker)	Almacena listas negras de tokens JWT, control de tasa (*rate limiting*) y caché de catálogos académicos de solo lectura.
Base de Datos	PostgreSQL Administrado (Primary + Standby)	Motor relacional transaccional (ej. AWS RDS db.t4g.medium o DigitalOcean Managed DB) con réplica de respaldo para conmutación por error.
Almacenamiento Backups	Almacenamiento en Nube Fría (S3 / Wasabi / B2)	Destino externo para copias de seguridad encriptadas fuera del centro de datos principal (Regla 3-2-1).

P6-01

2. Canales de Comunicación y Controles de Seguridad

- **Canales y Protocolos:**
  - **Cliente a Edge / Balanceador:** Exclusivamente `HTTPS (TLS 1.3)` sobre el puerto `443`. Se fuerza redirección automática de HTTP (80) a HTTPS.
  - **Balanceador a Nodos de Backend:** Red Privada Virtual (VPC) aislada mediante `HTTP/2` o `TLS interno`.
  - **Backend a Base de Datos y Caché:** Conexiones cifradas en tránsito vía `PostgreSQL Wire Protocol con SSL (verify-full)` en puerto 5432 y `Redis RESP con autenticación de contraseña (AUTH)` en puerto 6379, restringidas a la subred privada.
- **Segmentación de Red y Firewall:**
  - La base de datos y la caché residen en una **Subred Privada** sin dirección IP pública.
  - Se configuran Grupos de Seguridad / Firewall (UFW/AWS Security Groups) para que el puerto de la base de datos (5432) solo acepte tráfico proveniente de las IPs internas de los nodos de la API.
- **Manejo de Contraseñas y Secretos:**
  - **Contraseñas de Usuarios:** Cifrado unidireccional irreversible mediante **Argon2id** (o **BCrypt** con factor de costo 12). Nunca se persisten ni viajan en texto plano.
  - **Secretos del Sistema:** Las llaves criptográficas JWT, credenciales de BD y conexiones externas no se alojan en el repositorio de código; se inyectan en tiempo de arranque mediante variables de entorno seguras gestionadas por un servicio centralizado (ej. AWS Secrets Manager, Doppler o Vault).
- **Gestión de Sesiones y Tokens (JWT):**
  - **Access Token:** Formato JWT de vida muy corta (15 minutos) almacenado solo en memoria del navegador del cliente (SPA).
  - **Refresh Token:** De vida más prolongada (7 días), almacenado en una cookie con atributos `HttpOnly`, `Secure` y `SameSite=Strict` para prevenir ataques de Cross-Site Scripting (XSS) y CSRF. Se aplica rotación de refresh tokens en cada renovación.

P6-02

3. Estrategia de Disponibilidad (Objetivo 99,5 %)

Un SLA del 99,5 % mensual permite un **margen máximo de indisponibilidad no planificada de $\approx 3,65$ horas al mes**. Se garantiza mediante:

1. **Redundancia Activo-Activo en Capa de Aplicación:**
   - Se despliegan dos instancias idénticas del backend detrás del balanceador de carga. Si una instancia falla o consume el 100% de memoria, el balanceador redirige el 100% del tráfico al nodo sobreviviente en menos de 5 segundos.
2. **Despliegues sin Interrupción (*Zero-Downtime Deployments*):**
   - Se utiliza la estrategia **Rolling Update** (o Blue/Green): se actualiza la Instancia 1 mientras la Instancia 2 atiende tráfico; una vez superado el *health check*, se conmuta y se actualiza la Instancia 2. Esto garantiza que las actualizaciones regulares no consuman el presupuesto de caída.
3. **Alta Disponibilidad en Base de Datos:**
   - Uso de un PostgreSQL con réplica de reserva sincrónica o semi-sincrónica (*Standby Hot-Replica*). En caso de fallo crítico en el nodo primario, el proveedor o el gestor de clúster promueve la réplica a primaria automáticamente (*Failover automático* en 60 a 120 segundos).

P6-03

4. Respaldo y Recuperación ante Desastres (Disaster Recovery)

Para ajustarse al **RNF-09** con métricas institucionales razonables:

- **RPO (Punto de Recuperación Objetivo):** ≤ 15 minutos (máxima pérdida tolerable de datos).
- **RTO (Tiempo de Recuperación Objetivo):** ≤ 2 horas (tiempo máximo para volver a estar operativos tras catástrofe).

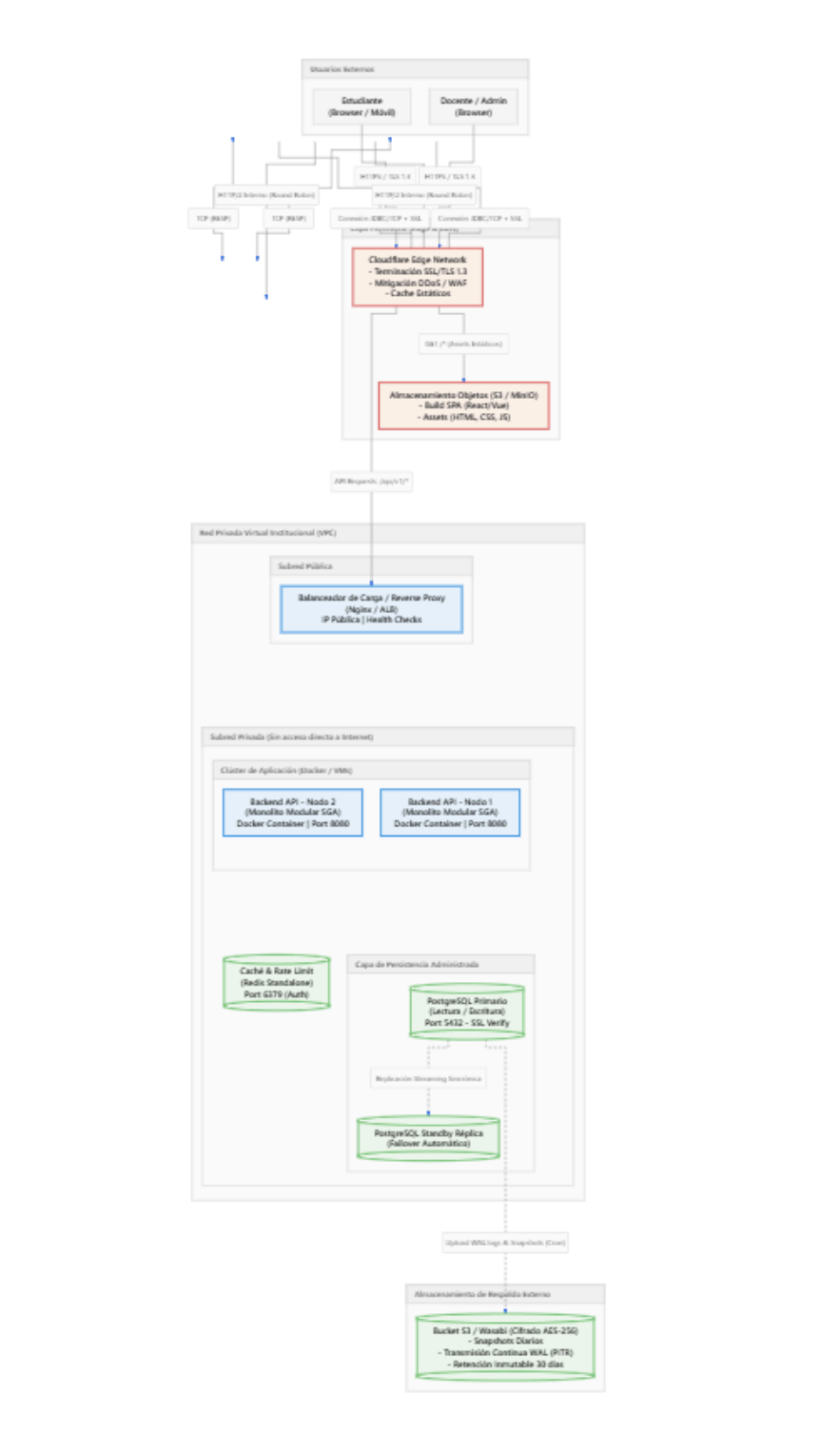
Procedimiento de Copias de Seguridad:

1. **Copias Completas Diarias:** A las 01:00 AM se ejecuta un volcado completo (`pg_dump` consistente o snapshot de volumen de BD) comprimido y cifrado con clave simétrica AES-256.
2. **Archivado Continuo de WAL (*Write-Ahead Logging*):** Los registros binarios de transacciones (WAL) se transmiten continuamente cada 10 minutos hacia un bucket de almacenamiento de objetos externo (S3/Wasabi). Esto permite **Recuperación en el Punto en el Tiempo (*Point-in-Time Recovery - PITR*)**.
3. **Inmutabilidad y Retención:** Política de retención de 30 días con política *Object Lock* (WORM) para evitar borrado accidental o por ransomware.

Procedimiento de Restauración:

- Script automatizado (`restore_db.sh`) probado trimestralmente en un entorno de pruebas (*staging*). Descarga el último snapshot diario, lo aplica sobre una nueva instancia de PostgreSQL y reaplica los archivos WAL hasta el minuto previo al fallo.

P6-04


Usuarios Internos
Estudiante
(Browser / Móvil)
Docente / Admin
(Browser)
Cloudflare Edge Network
- Terminación SSL/TLS 1.3
- Mitigación DDoS / WAF
- Cache Estáticos
Almacenamiento Objetos (S3 / MinIO)
- Build SPA (React/Vue)
- Assets (HTML, CSS, JS)
Red Privada Virtual Institucional (VPC)
Subred Pública
Balanceador de Carga / Reverse Proxy
(Nginx / ALB)
IP Pública | Health Checks
Subred Privada (Sin acceso directo a Internet)
Clúster de Aplicación (Docker / VMs)
Backend API - Nodo 2
(Monolito Modular SGA)
Docker Container | Port 8080
Backend API - Nodo 1
(Monolito Modular SGA)
Docker Container | Port 8080
Caché & Rate Limit
(Redis Standalone)
Port 6379 (Auth)
Capa de Persistencia Administrada
PostgreSQL Primario
(Lectura / Escritura)
Port 5432 - SSL Verify
PostgreSQL Standby Réplica
(Failover Automático)
Bucket S3 / Wasabi (Cifrado AES-256)
- Snapshots Diarios
- Transmisión Continua WAL (PITR)
- Retención Inmutable 30 días

P6-05

P7 · Revisión crítica

Ahora actúa como evaluador externo y critica la arquitectura que propusiste en esta conversación.

1. ¿Qué requerimientos funcionales o no funcionales quedaron sin cubrir o cubiertos de forma débil?
2. ¿Qué puntos únicos de falla existen?
3. ¿Qué riesgos de seguridad y privacidad ves?
4. ¿Cuáles son los límites de escalabilidad?
5. ¿Qué decisiones requieren validación adicional y cómo se validarían (pruebas de carga, pruebas de concurrencia, simulacros de restauración)? Sé concreto y no repitas lo que ya dijiste

P7-01

Como evaluador externo independiente, presento la auditoría técnica y crítica constructiva a la arquitectura propuesta:

---

1. Requerimientos Sin Cubrir o Cubiertos Débilmente

- **RF-05 (Prerrequisitos y Correquisitos Académicos):** El modelo de datos y el flujo de matrícula ignoraron por completo las dependencias entre asignaturas (árbol de prelaciones). No hay entidades ni validaciones que impidan a un estudiante inscribir "Cálculo II" sin haber aprobado "Cálculo I".
- **RNF-06 (Accesibilidad WCAG 2.1 AA):** Se mencionó como directriz de diseño, pero carece de soporte arquitectónico: no se definieron herramientas de verificación continua en el pipeline de CI/CD (ej. pruebas automatizadas con *Axe-core* o *Lighthouse CI*), dejando el cumplimiento a la disciplina del desarrollador de Front-End.
- **RNF-10 (Trazabilidad de Requerimientos a Código):** Quedó como una declaración de intenciones. No se especificó un estándar de *Architecture Decision Records* (ADR), versionado semántico de contratos OpenAPI ni etiquetado de commits vinculado a historias/tickets.
- **RF-06 (Gestión de Infraestructura de Aulas vs. Cupos):** El modelo asume que el cupo es solo una propiedad del `Grupo`, ignorando la capacidad física de la entidad `Aula`. Un grupo podría tener configurado un cupo de 40 estudiantes asignado a un aula física con capacidad máxima para 25.
- **RF-10 (Generación Pesada de Reportes):** El módulo de reportes lee directamente del motor de persistencia principal. En fechas críticas de cierre de actas, la generación concurrente de reportes pesados en PDF/Excel degradará el pool de conexiones transaccionales.

P7-02

2. Puntos Únicos de Falla (SPOFs)

1. **Balanceador de Carga Único en DMZ:** Aunque se propusieron dos nodos de API, solo se ilustró un único nodo de Nginx/HAProxy. Si este balanceador sufre un fallo de proceso, agotamiento de *file descriptors* o kernel panic, todo el sistema queda inaccesible (violando el RNF-03 de 99,5 %).
2. **Instancia Única de Redis (Standalone):** Si el contenedor de Redis falla:
   - El control de tasa (*rate limiting*) queda inoperativo.
   - La lista negra de JWTs se pierde en memoria, dejando vulnerables los tokens revocados.
   - Si la API depende de Redis para cachear catálogos, se producirá un efecto avalancha (*thundering herd*) contra PostgreSQL.
3. **Pérdida de Auditoría por *EventBus* en Memoria:** Si el nodo de la API sufre una caída de tensión o un error de memoria (*OOM Kill*) inmediatamente después de confirmar la transacción en la base de datos pero antes de que el *EventBus* entregue el evento a la cola asíncrona, **la pista de auditoría de esa matrícula se pierde definitivamente**, violando el RF-12.

P7-03

3. Riesgos de Seguridad y Privacidad

- **Vulnerabilidad del Log de Auditoría ante el Administrador de BD (DBA):** La tabla `audit.logs` reside en la misma base de datos relacional. Un DBA con privilegios de superusuario o un atacante que logre una inyección SQL a nivel administrativo puede ejecutar `UPDATE` o `DELETE` sobre las pistas de auditoría para encubrir alteraciones de calificaciones. **No es un almacenamiento WORM criptográficamente demostrable**.
- **Ambigüedad en Usuarios con Roles Múltiples:** En el modelo, `UsuarioRol` es M:N. Si un usuario es simultáneamente `Docente` de un departamento y `Estudiante` de posgrado, el diseño no contempla un mecanismo de **Contexto de Sesión Activo**. Un estudiante-docente podría interactuar con endpoints de calificaciones sin una validación estricta del perfil con el que inició la sesión actual.
- **Falta de Anonimización en Calificaciones (RNF-11):** Los endpoints exponen directamente la relación `EstudianteId` ↔ `Nota`. No se contemplaron esquemas de calificación a ciegas (*blind grading*), lo que expone a la institución a riesgos de sesgo o filtración de datos sensibles de rendimiento académico.

P7-04

4. Límites de Escalabilidad

- **Contención Extrema en el Cerrojo de Fila (`SELECT ... FOR UPDATE`):**
  - *El cuello de botella:* Si se abren las matrículas a las 08:00 AM y 250 de los 300 usuarios intentan matricularse simultáneamente en la misma asignatura popular, las 250 transacciones se serializan una a una sobre el mismo registro de la tabla `Grupo`.
  - *Consecuencia:* Aunque la latencia de una transacción individual sea de 15 ms, una cola de 200 transacciones esperando por el mismo lock acumulará demoras de más de 3 segundos, disparando *timeouts* y superando el límite del **RNF-02**.
- **Agotamiento del Pool de Conexiones de Base de Datos:**
  PostgreSQL utiliza un modelo de un proceso por conexión (`fork`). Dos instancias de API abriendo conexiones concurrentes contra un motor moderado pueden agotar rápidamente el parámetro `max_connections` (típicamente 100-200 conexiones en servidores modestos) si no se interpone un *connection pooler* transaccional como **PgBouncer**.
- **Techo Vertical del Monolito:** Toda la carga (escrituras críticas de matrícula, subida de notas, analítica pesada y autenticación) compite por la CPU y el I/O del mismo motor de base de datos primario.

P7-05

5. Decisiones que Requieren Validación Adicional y Planes de Prueba

Decisión Arquitectónica Crítica	Riesgo Asociado	Plan de Validación Propuesto
Bloqueo Pesimista en Cupos (`FOR UPDATE`)	*Deadlocks*, colas de espera excesivas y agotamiento de conexiones en picos de alta demanda.	**Prueba de Concurrencia Extrema:** Ejecutar una prueba con herramientas como *k6* o *Gatling* simulando a 300 usuarios disparando la matrícula sobre un único grupo con 5 cupos restantes en una ventana de 2 segundos. Medir contención, latencia en P95/P99 y verificar matemáticamente cero sobrecupos.
Resiliencia de Auditoría vía EventBus	Pérdida de eventos de auditoría por caídas abruptas del proceso.	**Prueba de Caos / Falla Inyectada:** Enviar un flujo continuo de matrículas y matar el proceso del contenedor (`kill -9`) en momentos aleatorios. Comparar registros de `DetalleMatricula` contra registros en `audit.logs`. Si hay discrepancias, rediseñar usando el **Patrón Outbox Transaccional**.
RPO ≤ 15 min y RTO ≤ 2 h	Estimaciones teóricas no comprobadas sobre restauración de datos.	**Simulacro de Desastre (Fire Drill):** Corromper intencionalmente la base de datos en un entorno réplica, activar cronómetro y ejecutar el procedimiento de restauración desde el bucket externo aplicando Snapshots + WAL streaming hasta el último minuto.
Dimensionamiento para 300 Concurrentes	Sobredimensionamiento o subdimensionamiento de recursos de cómputo en servidores moderados.	**Prueba de Carga y Capacidad (Stress Testing):** Modelar el comportamiento típico (70% lecturas de horarios/notas, 20% matrículas, 10% reportes) y elevar paulatinamente la concurrencia de 50 a 500 usuarios. Identificar el punto de degradación donde el P95 supere los 3 segundos.

P7-06

Anexo C. Texto de los prompts

P1 · Análisis del problema

```
Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión
Académica (SGA) para una institución de educación superior. Necesito analizarlo antes de diseñar
la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes,
programas académicos, asignaturas, grupos, matrículas, calificaciones y horarios. Hoy parte de
la información se maneja en archivos y aplicaciones independientes, lo que genera duplicidad de
datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura,
mantenible, escalable y accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante;
restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas
(código, nombre, créditos, semestre o nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados,
horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y
evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus
grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y
calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros
por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o
estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de
calificaciones (usuario, fecha, acción, registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300
usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos
programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras,
contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y
preservan consistencia ante errores o solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y
evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

TAREA:
1. Identifica los actores principales y secundarios.
```

```
2. Propón supuestos razonables donde el caso no es explícito.
3. Lista las restricciones técnicas y del negocio.
4. Extrae las reglas de negocio implícitas en los requerimientos (por ejemplo, sobre cupos,
matrículas duplicadas y quién puede modificar calificaciones).
5. Clasifica cada RNF según el atributo de calidad que representa.
Responde en español, con tablas cuando ayuden.
```

P2 · Propuesta de estilo arquitectónico

```
Con base en el análisis anterior, propón el estilo arquitectónico para el SGA.
Compara al menos tres opciones: arquitectura en capas, monolito modular y microservicios.
Para cada una evalúa: complejidad de desarrollo y operación, rendimiento con 300 usuarios
concurrentes, disponibilidad del 99,5 %, escalabilidad, mantenibilidad y consistencia en la
matrícula ante solicitudes simultáneas.
Al final recomienda una opción y justifícala frente a los requerimientos no funcionales.
Responde en español con una tabla comparativa.
```

P3 · Componentes y vista lógica

```
Para la arquitectura que recomendaste, describe la vista lógica:
los módulos o componentes principales, la responsabilidad de cada uno, las interfaces que
exponen (por ejemplo, endpoints REST principales) y las dependencias entre ellos.
Debe quedar claro cómo se organizan autenticación y autorización, gestión académica, matrículas,
calificaciones, reportes y auditoría.
Genera además el diagrama en sintaxis Mermaid.
```

P4 · Modelo de datos conceptual

```
Propón el modelo de datos conceptual del SGA con las entidades: Usuario, Rol, Estudiante,
Docente, Programa, Asignatura, PeriodoAcadémico, Grupo, Matrícula y Calificación, más las que
consideres necesarias (por ejemplo, para horarios o auditoría).
Indica los atributos principales y las cardinalidades de cada relación, y explica las
restricciones que evitan matrículas duplicadas.
Genera el diagrama entidad-relación en sintaxis Mermaid (erDiagram).
```

P5 · Secuencia de la matrícula

```
Modela el diagrama de secuencia del proceso de matrícula de un estudiante en un grupo.
Incluye los participantes (estudiante o coordinador, interfaz web, API, módulos internos, base
de datos y auditoría), el orden de los mensajes, la validación de cupo, la validación de
matrícula duplicada y qué pasa si dos solicitudes llegan al mismo tiempo por el último cupo.
Genera el diagrama en sintaxis Mermaid (sequenceDiagram).
```

P6 · Despliegue, seguridad y respaldo

```
Propón el modelo de despliegue del SGA: nodos o servicios donde se ejecutan la interfaz web, la
API, la base de datos y los servicios complementarios.
Indica los canales de comunicación, los controles de seguridad (HTTPS, firewall, manejo de
contraseñas, tokens), cómo se logra la disponibilidad del 99,5 % y cómo serían las copias de
seguridad y la restauración.
```

```
Considera que es una institución con presupuesto moderado. Genera el diagrama en sintaxis
Mermaid.
```

P7 · Revisión crítica de la propia propuesta

```
Ahora actúa como evaluador externo y critica la arquitectura que propusiste en esta
conversación.
1. ¿Qué requerimientos funcionales o no funcionales quedaron sin cubrir o cubiertos de forma
débil?
2. ¿Qué puntos únicos de falla existen?
3. ¿Qué riesgos de seguridad y privacidad ves?
4. ¿Cuáles son los límites de escalabilidad?
5. ¿Qué decisiones requieren validación adicional y cómo se validarían (pruebas de carga,
pruebas de concurrencia, simulacros de restauración)?
Sé concreto y no repitas lo que ya dijiste.
```